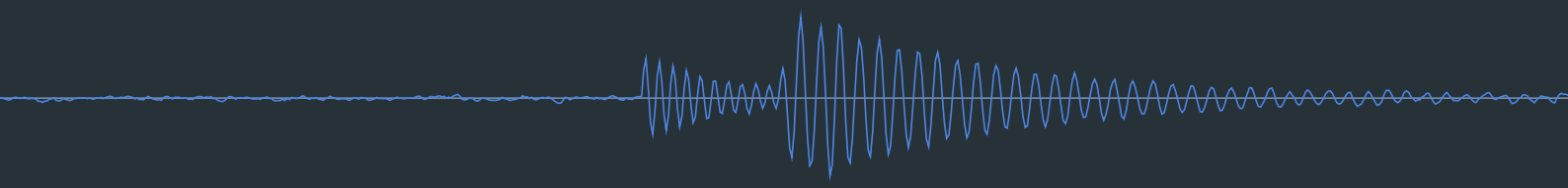
RS3D

Raspberry Shake 3D



Theory and Algorithms Manual

The mathematics and signal processing behind every function of the RS3D application

Edition 1.0 • October 2026 • for rs3d-app 0.1 and later

Companion to the RS3D User's Guide. All figures are generated by the application's own code from a reproducible synthetic recording.

Preface

RS3D is an application for the Raspberry Shake 3D seismograph. Its *User's Guide* explains how to use it; this manual explains *why it works*. It documents, one function at a time, the signal-processing theory that the application relies on, how that theory was turned into code, and where the code deliberately or unavoidably departs from the textbook version.

The manual is written for three kinds of reader: an observer who wants to trust (or question) what a spectrogram or an alert is telling them, a student who wants a worked example of applied seismological signal processing, and a developer who is about to change one of the algorithms and needs the assumptions written down.

What is new here, and what is not. Nothing in this manual is new science. Every method (Butterworth filtering, Welch spectra, short-time Fourier analysis, the STA/LTA trigger) is standard and is cited. What the manual adds is precision about *this* implementation: the exact windows, constants, tolerances and edge-case rules, each traceable to a source file of the application.

How the chapters are organised. Chapters follow the path of a sample through the application, from the sensor (Chapters 2 and 3) through filtering and spectral analysis (Chapters 4–7) and data quality (Chapter 8) to visualisation, detection and real-time handling (Chapters 9–11). Chapter 12 describes how the code is organised, and the appendices collect derivations and every numerical constant used by the application.

How to read the boxes

Key idea boxes state a result in one or two sentences.

In RS3D boxes say exactly what the application does, naming the source file (paths are relative to `rs3d-app/src/rs3d/`). Where the application departs from the textbook method this is stated there, never left implicit.

Reproducibility. Every plot is produced by one script, `make_figures.py`, which calls the application's own functions (not re-implementations) on a seeded synthetic recording of one day at a fictitious station (`AM.DEMO1.00`). The recording contains a local earthquake, a distant earthquake, two man-made noise bursts and two data gaps, so each algorithm can be shown on a case whose true answer is known.

Status of the alert chapter. The event-detection theory of Chapter 10 is complete. The design of the application's alert pipeline in Section 10.9 is a *design proposal*: that feature is not yet implemented, and the proposal will be revised when it is.

Verification of references. Two of the key references (Trnkoczy 2012 and McNamara and Buland 2004) were checked against online bibliographic records while this edition was written. The remaining entries are well-known works cited from the authors' knowledge; their volume and page numbers should be confirmed against the originals before the references are reused in a formal publication. The description of rsudp's alert defaults in Chapter 10 is deliberately hedged for the same reason: no authoritative list of defaults could be located.

RS3D project, October 2026

Notation

Symbol	Meaning	Unit
$x(t)$, $x[n]$	continuous signal; its samples $x[n] = x(n\,\Delta t)$	counts or m/s
f_s, Δt	sampling frequency, sampling interval $\Delta t = 1/f_s$	Hz, s
f_N	Nyquist frequency, $f_N = f_s/2$	Hz
N	number of samples in a window or record	–
T	duration $T = N\,\Delta t$	s
$X[k]$	discrete Fourier transform of $x[n]$	counts
f_k	frequency of bin k, $f_k = k f_s/N$	Hz
Δf	frequency resolution (bin spacing) $\Delta f = f_s/N = 1/T$	Hz
$w[n]$	window (taper) function	–
$H(f)$	transfer function of a filter	–
$T(f)$	instrument response, counts per unit of ground velocity	counts/(m/s)
$S_{xx}(f)$	one-sided power spectral density	counts^2/Hz
STA, LTA	short- and long-term averages	counts^2
R	STA/LTA ratio	–
j	imaginary unit	–

Contents

Preface iii

Notation v

1 Introduction 1
- 1.1 What the application does . . . 1
- 1.2 Where each feature is explained . . . 2
- 1.3 Conventions . . . 2

2 Seismic data fundamentals 3
- 2.1 From ground motion to numbers . . . 3
- 2.2 Sampling and the Nyquist frequency . . . 3
- 2.3 Channel naming . . . 4
- 2.4 miniSEED and the day archive . . . 4
- 2.5 Time . . . 5

3 Instrument response and units 7
- 3.1 Why counts are not ground motion . . . 7
- 3.2 Linear systems and the transfer function . . . 7
  - 3.2.1 A geophone as a second-order system . . . 7
  - 3.2.2 The complete chain . . . 8
- 3.3 Removing the response . . . 8
- 3.4 The DC offset . . . 9

4 Digital filtering 11
- 4.1 Frequency response of a digital filter . . . 11
- 4.2 The Butterworth filter . . . 11
- 4.3 Zero-phase filtering . . . 11
- 4.4 Edge effects . . . 12
- 4.5 Choosing a band . . . 13
- 4.6 Summary . . . 14

5 Spectral analysis 15
- 5.1 The discrete Fourier transform . . . 15
- 5.2 Windowing and spectral leakage . . . 15
- 5.3 Scaling the amplitude spectrum . . . 16
- 5.4 Amplitude spectrum versus power spectral density . . . 17
- 5.5 Variance of a spectrum estimate and why averaging helps . . . 18

6 The spectrogram 21
6.1 The short-time Fourier transform . . . 21
6.2 The time–frequency trade-off . . . 21
6.3 The window used by RS3D . . . 22
6.4 Choosing the window length: the 360 rule . . . 23
6.5 Gaps . . . 24
6.6 From numbers to pixels . . . 24
6.7 Reading a spectrogram . . . 25

7 The noise spectrum 27
7.1 Definition . . . 27
7.2 Relation to the McNamara–Buland PPSD . . . 27
7.3 What the curves tell you . . . 29

8 Data quality: gaps and completeness 31
8.1 Samples, spans and gaps . . . 31
8.2 Completeness . . . 31
8.3 Coverage: laptop, Shake, or nothing . . . 32
8.4 Why a gap is never filled . . . 33

9 Visualisation algorithms 35
9.1 Reducing a signal for display . . . 35
9.1.1 Stride decimation and why it can lose a spike . . . 35
9.1.2 Min/max decimation . . . 35
9.2 The helicorder . . . 36
9.2.1 Geometry . . . 37
9.2.2 Removing the offset . . . 37
9.2.3 The amplitude scale . . . 38
9.2.4 From a click to a time . . . 38
9.3 The detail view . . . 38

10 Event detection and alerts 41
10.1 The detection problem . . . 41
10.2 The characteristic function . . . 41
10.3 The classic STA/LTA . . . 42
10.3.1 The warm-up period . . . 42
10.4 The recursive STA/LTA . . . 42
10.5 Setting the parameters . . . 44
10.5.1 Statistical meaning of the ratio . . . 45
10.6 Behaviour on the demo day . . . 45
10.7 Failure modes and how to reduce them . . . 47
10.7.1 Man-made noise looks like an earthquake . . . 47
10.7.2 Other failure modes . . . 47
10.8 Latency . . . 49

10.9 Design of the RS3D alert pipeline 49

11 Real-time data handling **53**
11.1 SeedLink 53
11.2 The ring buffer 53
11.3 Keeping the link alive 54
11.4 Downloading the archive 55
11.5 Latency 55

12 Software architecture **57**
12.1 Two layers 57
12.2 Where each algorithm lives 57
12.3 Reproducibility by construction 58
12.4 Testing 58
12.5 Configuration and storage 59

A Derivations **61**
A.1 Scaling of the amplitude spectrum 61
A.2 Hann window constants 61
A.3 Zero-phase filtering 61
A.4 The recursive averages 62
A.5 Detection delay 62
A.6 The F distribution of the ratio 62
A.7 Error of min/max decimation 63

B Parameter reference **65**

References **67**

1 Introduction

1.1 What the application does

RS3D reads the three-component ground-velocity signal of a Raspberry Shake 3D, keeps it on the laptop, and turns it into pictures and numbers an observer can act on. Seen from the signal-processing side its work divides into five tasks:

1. **Acquire**: receive samples in real time (SeedLink) and copy the Shake's day files (SFTP).
2. **Condition**: convert counts to ground velocity and remove unwanted frequencies.
3. **Analyse**: estimate spectra, spectrograms and a noise summary; measure data completeness.
4. **Display**: reduce millions of samples to a few thousand points per plot without losing what matters.
5. **Detect**: decide, sample by sample, whether the ground is moving more than usual (alerts).

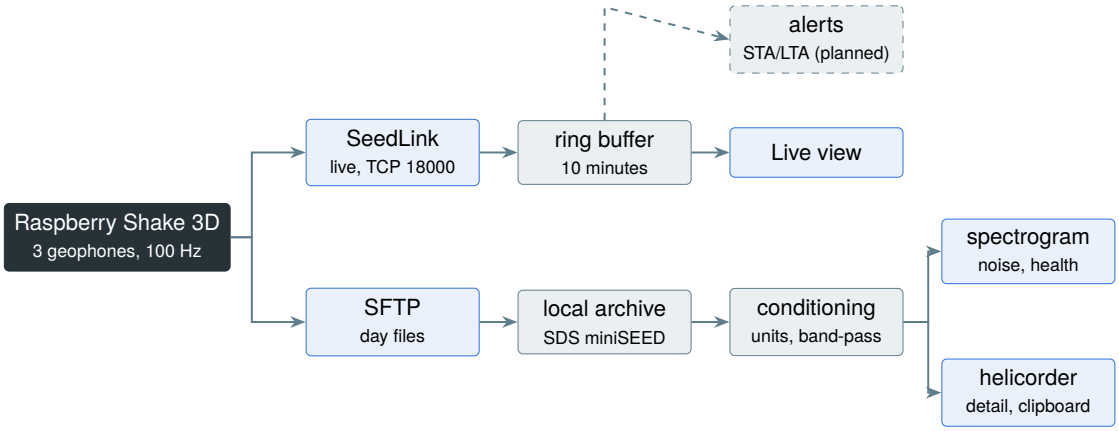


Figure 1.1: Data flow of RS3D. Solid boxes exist today; the dashed alert stage is designed in Section 10.9.

Table 1.1: Application features and the chapters that explain them.

Feature	Theory used	Chapter
Counts / m/s switch	instrument response, deconvolution	3
Band-pass (Data, Live, Helicorder)	Butterworth, zero-phase filtering	4
Spectra mode, clipboard spectrum	discrete Fourier transform, windows	5
Spectrogram	short-time Fourier transform	6
Noise spectrum	percentile summary of spectra	7
Coverage bar, Data health	gaps, completeness	8
Live and helicorder traces, click-to-time	min/max decimation, row geometry	9
Alerts	STA/LTA trigger	10
Live link, reconnection	SeedLink, buffering, back-off	11

1.2 Where each feature is explained

Table 1.1 links every function a user can see to the chapter that derives it.

1.3 Conventions

Time. All data are stored and processed in UTC. The application can *display* times in the laptop's local zone, but that only changes labels; no sample is shifted. Times inside the code are epoch seconds (`UTCDateTime.timestamp`).

Amplitude units. Unless stated otherwise amplitudes are in *counts*, the integers produced by the Shake's analogue-to-digital converter. Chapter 3 explains the conversion to ground velocity in m/s.

Decibels. A power ratio of P is $10\log_{10} P$ dB; an amplitude ratio of A is $20\log_{10} A$ dB. The application's spectrograms use $10\log_{10}$ of power spectral density in counts^2/Hz, written "dB re. 1 count^2/Hz".

Software versions

The application was developed against ObsPy 1.5.1 (Beyreuther et al. 2010; Krischer et al. 2015), SciPy 1.x and NumPy. Where behaviour depends on a library default (for example SciPy's spectrogram window or ObsPy's response removal), the default is stated and was checked directly against the installed library.

2 Seismic data fundamentals

2.1 From ground motion to numbers

A seismometer converts ground motion into an electrical signal. The Raspberry Shake 3D carries three orthogonal geophones, nominally with a natural frequency of $f_0 = 4.5$ Hz, one vertical (Z) and two horizontal (N–S and E–W). A geophone is a coil suspended in the field of a magnet fixed to the case. When the ground, and with it the case, moves, the coil lags behind by inertia, and the relative velocity between coil and magnet induces a voltage proportional to that *relative velocity*. Above its natural frequency the relative velocity of the coil equals the ground velocity, so the voltage is proportional to ground velocity in the pass band (Chapter 3 derives the transfer function; Havskov and Alguacil (2016) treat the physics in depth).

The analogue voltage is then amplified and digitised by an analogue-to-digital converter (ADC). The integers it produces are called *counts*. A count is not a physical unit: one count corresponds to a fixed number of volts, and, through the sensor, to a frequency-dependent velocity. The conversion is the subject of Chapter 3.

2.2 Sampling and the Nyquist frequency

The ADC measures the signal at regular intervals Δt, giving samples $x[n] = x(n\,\Delta t)$. The Raspberry Shake 3D samples at $f_s = 100$ samples per second, so $\Delta t = 10$ ms. The sampling theorem states that a band-limited signal can be reconstructed exactly from its samples if and only if it contains no energy at or above the *Nyquist frequency*

$$f_N = \frac{f_s}{2} = 50 \text{ Hz}. \tag{2.1}$$

Energy above f_N does not vanish when sampled: it *aliases*, folding back into the band below f_N. A sinusoid of frequency f with $f_N < f < f_s$ produces exactly the same samples as one of frequency $f_s - f$ (figure 2.1). Aliasing cannot be undone after sampling, so digitisers apply an analogue and digital *anti-alias filter* before and during decimation; its roll-off is visible as a fall in the spectra of RS3D near f_N (Chapter 7). Consequently the top few hertz of any RS3D spectrum are attenuated by the instrument, not by the ground.

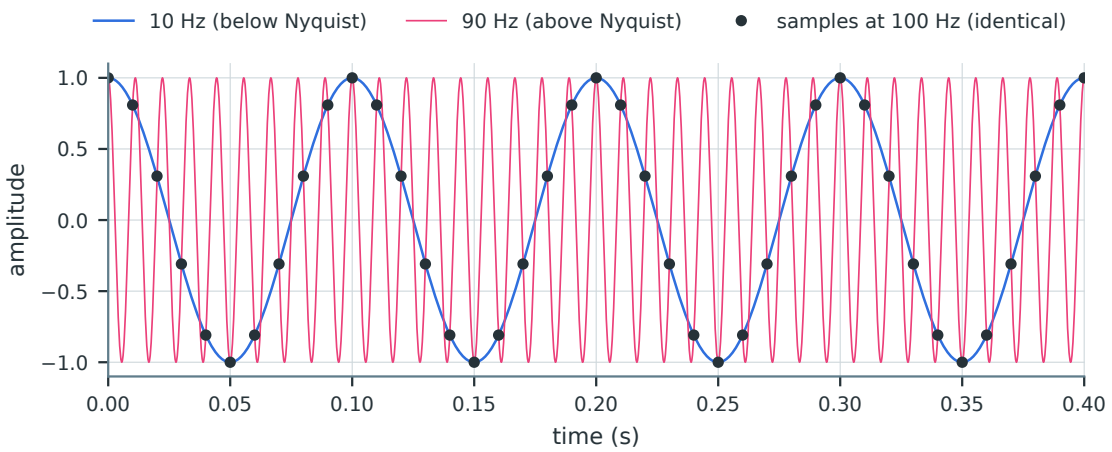


Figure 2.1: Aliasing. At $f_s = 100$ Hz a 90 Hz cosine (pink) and a 10 Hz cosine (blue) have identical samples (dots). Because $90 = f_s - 10$, the faster signal is indistinguishable from the slower one after sampling.

Key idea

Everything the application shows lies below $f_N = 50$ Hz, and the last few hertz below it are shaped by the Shake's anti-alias filter.

2.3 Channel naming

Every stream of samples is identified by four codes, the SEED identifier (International Federation of Digital Seismograph Networks 2012):

network	station	location	channel
AM	R77A7	00	EHZ

`AM` is the Raspberry Shake network, `R77A7` the station, `00` the location code. The three-letter channel code is itself structured: `E` is the *band* code (an extremely short-period instrument sampled at 80 Hz or more), `H` the *instrument* code (a high-gain seismometer), and the last letter the *orientation*: `Z` vertical, `N` north–south, `E` east–west. An RS3D therefore serves `EHZ`, `EHN` and `EHE`. The application sorts channels vertical-first (`Z`, `N`, `E`) so that the default channel is always the vertical.

2.4 miniSEED and the day archive

Seismic samples are exchanged as *miniSEED*, a sequence of fixed-length records (512 bytes is typical for SeedLink). Each record carries the four identifier codes, the time

of its first sample, the sampling rate, and a block of samples stored as first differences compressed with the Steim algorithm. Because consecutive samples of a smooth signal differ by small amounts, the compression is lossless and efficient. One channel-day is 8.64 million samples, about 35 MB as raw 32-bit integers; the Shake's compressed archive for all three channels takes about 85 MB per day (the figure quoted in the application's own storage help).

A *gap* is a place where a record's start time does not continue the previous record. Readers such as ObsPy split a stream at gaps into separate *traces*; the application keeps them separate (and shows them as breaks or blank columns) rather than inventing data. How gaps are measured is the subject of Chapter 8.

Files are arranged in the *SeisComP data structure* (SDS), one file per channel per day:

```
<root>/YEAR/NET/STA/CHAN.D/NET.STA.LOC.CHAN.D.YEAR.DOY
```

where `DOY` is the day of the year, zero-padded to three digits. For example the vertical channel of 15 October 2026 is stored in `2026/AM/R77A7/EHZ.D/AM.R77A7.00.EHZ.D.2026.288`. The Shake keeps the same layout under `/opt/data/archive`, and the application mirrors it on the laptop, so a file downloaded from the Shake can be used unchanged.

In RS3D

`core/store.py` implements the day-file layout (`day_path`, `days`, `load`); `import_file` splits any miniSEED file into day files, merging with data already present so that importing the same file twice changes nothing. `core/archive.py` copies day files from the Shake: it downloads into a temporary `.part` file and renames it only when complete, so an interrupted transfer never looks like a finished day, and it always re-fetches the newest remote day because the Shake is still writing to it.

2.5 Time

Sample times are UTC. The time of sample n of a record starting at t_0 is $t_0 + n/f_s$. The accuracy of t_0 depends entirely on the Shake's clock, which is kept right by the network time protocol; an offset of 100 ms is already visible when two stations are compared. The Status page therefore flags a clock offset above 100 ms as a warning and above 1000 ms as an error (`core/status.py`). Leap seconds are not modelled: timestamps are POSIX-style seconds, which is adequate for the minute-to-second resolution the application targets.

3 Instrument response and units

3.1 Why counts are not ground motion

A record in counts mixes two things: the ground motion and the way the instrument responds to it. The same ground velocity at 0.1 Hz and at 10 Hz produces very different counts, because a geophone responds weakly below its natural frequency. To compare records from different instruments, or to report ground velocity, the instrument's effect must be removed. This is done with the *instrument response*.

3.2 Linear systems and the transfer function

Over its operating range a seismometer behaves as a linear, time-invariant system. If the ground velocity is $v(t)$ and the digitised output is $d(t)$ (in counts), then in the frequency domain

$$D(f) = T(f)\, V(f), \tag{3.1}$$

where V and D are the Fourier transforms of v and d and $T(f)$ is the complex *transfer function*, in counts per (m/s). Its magnitude $|T|$ says how much each frequency is amplified; its phase says how much it is delayed.

3.2.1 A geophone as a second-order system

The mechanical part of a geophone is a damped oscillator. With natural angular frequency $\omega_0 = 2\pi f_0$ and fractional damping h the velocity transfer function in the Laplace variable $s = \mathrm{j}\omega$ is

$$T_g(s) = G\,\frac{s^2}{s^2 + 2h\,\omega_0\, s + \omega_0^2}, \tag{3.2}$$

where G is the generator constant (volts per m/s) in the flat band. Equation (3.2) has two zeros at $s = 0$ and two poles at $s = \omega_0(-h \pm \mathrm{j}\sqrt{1-h^2})$. Figure 3.1 shows its magnitude and phase for $f_0 = 4.5$ Hz.

Three properties matter for what follows.

1. **Flat above f_0.** For $f \gg f_0$, $T_g \to G$: the output follows ground velocity directly.

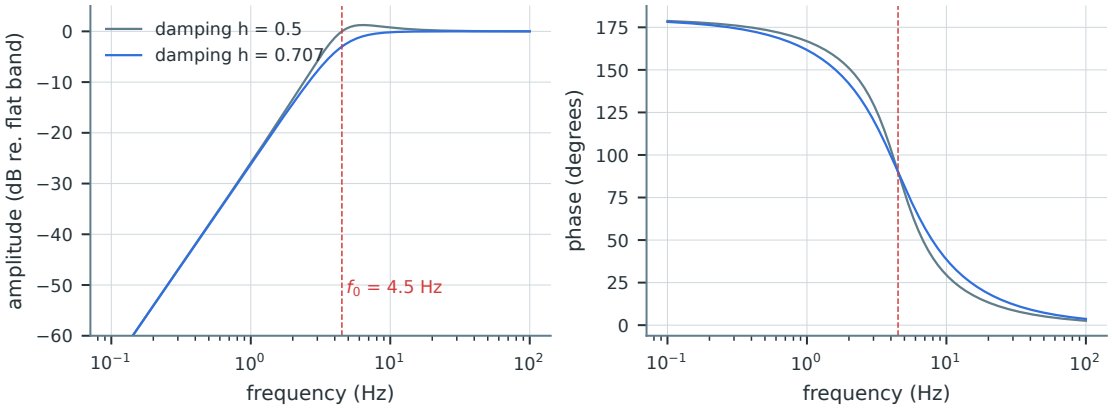


Figure 3.1: Magnitude and phase of the velocity response (3.2) of a 4.5 Hz geophone for two damping values. Illustrative only: the real response of a given Shake comes from its StationXML file.

2. **Falling as f^2 below f_0.** The double zero at the origin makes the response fall at 40 dB per decade (a factor 100 in amplitude for each factor 10 in frequency).

3. **Zero at $f = 0$.** A constant (DC) input produces no output from the mechanical part.

3.2.2 The complete chain

The digitiser adds its own stages: amplification, a digital anti-alias filter and decimation. The *overall* response is the product of the stage responses. Standards describe it as poles, zeros and a normalisation, and distribute it as *StationXML*. In the flat band the overall response reduces to a single number, the *sensitivity* Λ in counts per m/s.

3.3 Removing the response

Estimating the ground velocity means dividing by T in the frequency domain:

$$\hat{v}(t) = \mathcal{F}^{-1}\left[\frac{D(f)}{T(f)}\right]. \tag{3.3}$$

Equation (3.3) hides a numerical trap. Below f_0, $|T|$ becomes very small, so dividing by it *amplifies* whatever is there, which is mostly noise, and at $f = 0$ the division is by zero. Practical implementations therefore (i) remove the mean first, (ii) taper the record so that its ends do not create discontinuities, and (iii) limit the division with a *water level*: wherever $|T|$ falls below a chosen fraction of its maximum, it is replaced by that fraction.

In RS3D

The **counts**/**m/s** switch calls ObsPy's `remove_response(inventory, output="VEL")` in `core/export.py`, with ObsPy's defaults, which were read from the installed library: zero-mean removal on, a 5 % cosine taper, a water level of 60 dB, and no pre-filter. The response is a StationXML file downloaded once from the Raspberry Shake FDSN web service and cached in the data folder (`core/response.py`); with the file cached the switch works offline. Without a cached response and without Internet the Data page reports the error, and the Helicorder page reports it and returns to counts instead of showing an empty chart.

Velocity below the natural frequency

Because the response is deconvolved with a water level, ground velocity at frequencies well below 4.5 Hz is only as reliable as the signal-to-noise ratio there. Long-period phenomena, such as the ocean microseism near 0.2 Hz or the surface waves of distant earthquakes, are recoverable but noisier than the same signal on a broadband sensor.

3.4 The DC offset

Raw counts almost never oscillate around zero. Electronic offsets in the sensor and ADC add a constant, typically thousands of counts for a Shake. Because $T(0) = 0$, such a constant is *not* ground motion, and any processing that does not remove it has to cope with it:

- **Spectra.** An offset leaks into the lowest bins and, through window sidelobes, into higher ones. The application subtracts the mean before every spectrum (Chapter 5).
- **Filtering.** The filter route removes the mean before filtering (`detrend("demean")` in `core/export.py`).
- **The helicorder.** Plotting raw counts would shift every row by the offset and dominate the amplitude scale (figure 9.2 in Chapter 9); each row therefore has its median removed.

Key idea

Counts are an instrument-dependent integer with an arbitrary offset. Ground velocity needs the response; every display that does not use it must at least remove the mean.

4 Digital filtering

A filter keeps some frequencies and suppresses others. The application offers a band-pass filter on the Data, Live and Helicorder pages: slow drifts below the lower corner and electrical or traffic noise above the upper corner are removed, leaving the band in which the phenomenon of interest lives.

4.1 Frequency response of a digital filter

A linear filter acts on a sampled signal by convolution, $y[n] = \sum_m h[m]\,x[n-m]$. Its effect at frequency f is described by $H(f) = \sum_m h[m]\,e^{-\mathrm{j}2\pi f m/f_s}$, a complex number whose magnitude is the *gain* and whose angle is the *phase shift*. A band-pass filter has gain close to 1 between two *corner frequencies* $f_1 < f_2$ and gain close to 0 outside.

4.2 The Butterworth filter

Among the classical filter families, the Butterworth filter (Butterworth 1930) has the property that its magnitude is as flat as possible in the pass band. The magnitude of a Butterworth low-pass filter of order n and cut-off ω_c is

$$|H(\mathrm{j}\omega)|^2 = \frac{1}{1 + (\omega/\omega_c)^{2n}}. \tag{4.1}$$

At $\omega = \omega_c$ the gain is $1/\sqrt{2}$, that is -3 dB, for every order. Beyond the cut-off the gain falls by $20n$ dB per decade ($6n$ dB per octave), so a higher order gives a steeper edge. A band-pass filter is obtained from a low-pass prototype by the substitution $s \mapsto (s^2 + \omega_1\omega_2)/(s(\omega_2 - \omega_1))$, which doubles the number of poles: an order-n band-pass has $2n$ poles. Digital implementations realise the filter as a cascade of second-order sections (SOS) rather than one high-order polynomial, because the cascade is numerically far more stable.

4.3 Zero-phase filtering

A causal filter delays different frequencies by different amounts (phase distortion), which smears and shifts waveforms. A standard remedy is to filter the record forward, reverse

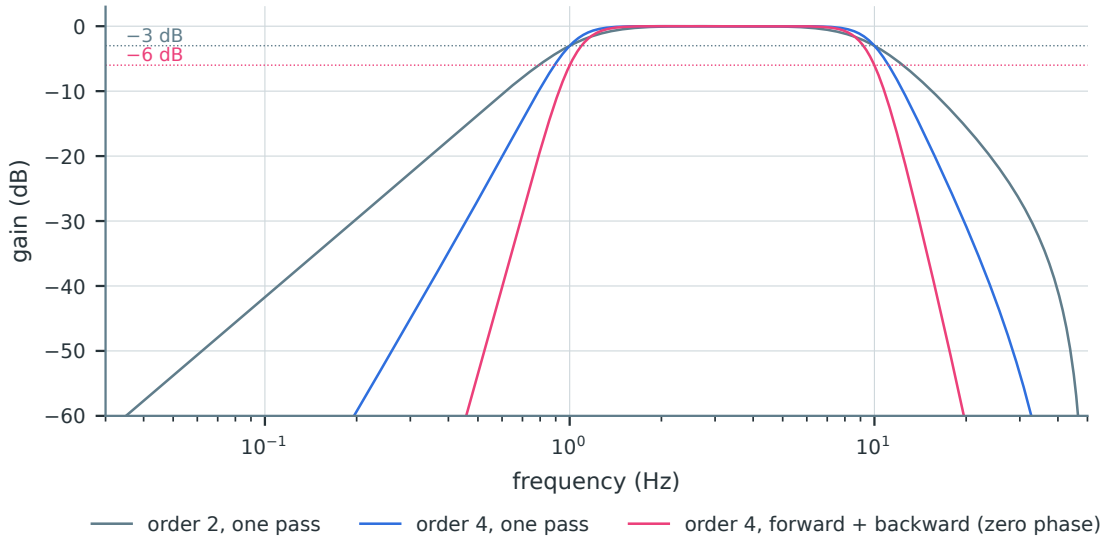


Figure 4.1: Gain of band-pass Butterworth filters with corners 1 and 10 Hz at $f_s = 100$ Hz: order 2 and order 4 in a single pass, and order 4 applied forward and backward (zero phase), whose gain in decibels is exactly twice that of one pass.

the output, filter it again, and reverse the result. If one pass has frequency response $H(f)$, the backward pass contributes $H^*(f)$ (the complex conjugate), so the combined response is

$$H_{zp}(f) = H(f)\,H^*(f) = |H(f)|^2, \tag{4.2}$$

a real, non-negative number: *zero phase shift*. Two consequences follow from (4.2):

1. **The gain in decibels doubles.** The −3 dB corner of a single pass becomes a −6 dB corner (figure 4.1), and the roll-off steepens to $12n$ dB per octave.
2. **The filter is non-causal.** The output at time t depends on input after t.

Non-causal filters move energy before the onset

A zero-phase filter smears the sharp start of an arrival *backwards* in time, as figure 4.2 shows: the filtered record contains oscillations before the true onset. Zero-phase filtering is therefore appropriate for looking at a waveform, but it must not be used to pick onset times or to drive a real-time trigger. The detector of Chapter 10 uses a causal filter for exactly this reason.

4.4 Edge effects

A filter has a memory: its output near the start of a record depends on samples that were never recorded. The first and last seconds of a filtered trace contain a *transient* and are

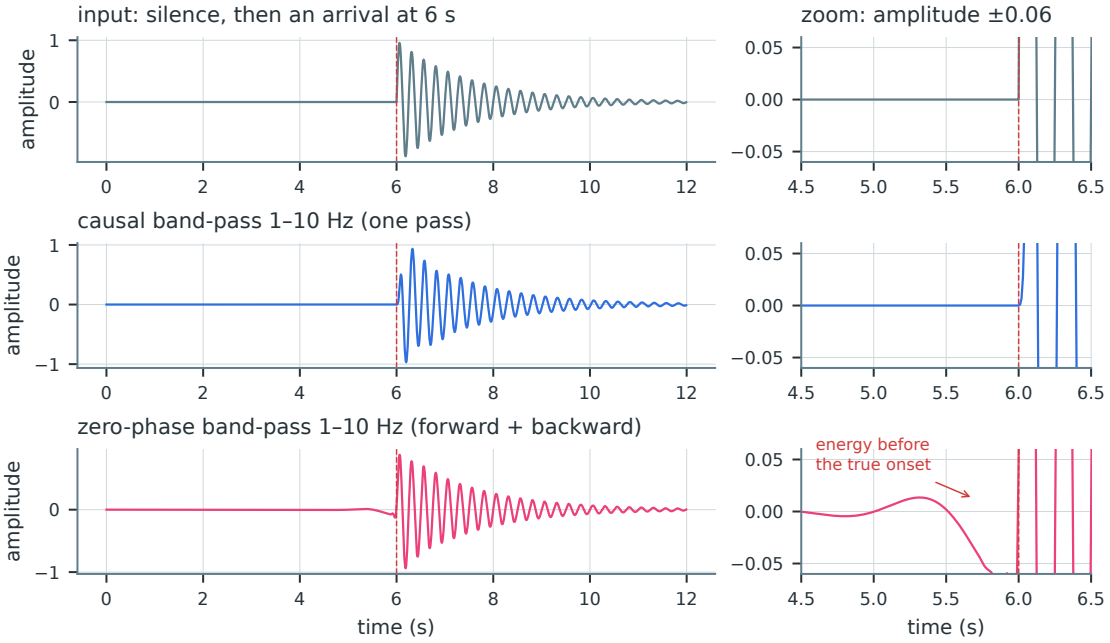


Figure 4.2: An ideal arrival at 6 s filtered by a 1–10 Hz band-pass, causally (one pass) and with zero phase. The causal output is exactly zero before the arrival; the zero-phase output rings before it (enlarged on the right). The zero-phase curve was computed with SciPy's padded `sosfiltfilt`.

Table 4.1: Typical pass bands for three-component short-period data.

Purpose	Band (Hz)	Comment
Local and regional earthquakes	1–10	the RS3D live default; rejects drift and most wind or traffic noise
Teleseismic body waves	0.5–3	the Shake's sensitivity falls quickly below its 4.5 Hz natural frequency
Microseism	0.1–0.5	strong everywhere; useful as a check of the instrument
Cultural noise	5–25	footsteps, traffic, machinery

less reliable. The effect is largest for narrow bands and low corners; a practical rule is to distrust about $3/f_1$ seconds at each end. The same holds at every *gap*: each contiguous piece of a trace is filtered on its own, so the edges of each piece carry a transient.

4.5 Choosing a band

Table 4.1 lists bands commonly used with a short-period sensor such as the RS3D. They are starting points, not rules; the right band depends on the noise at the site (Chapter 7).

In RS3D

RS3D uses two different implementations, deliberately.

- **Data and Helicorder pages, exports** (`core/export.py`). ObsPy's `filter("bandpass", freqmin, freqmax, corners=4, zerophase=True)` after `detrend("demean")`. Reading the library source and comparing outputs shows that ObsPy implements `zerophase` as two passes of an SOS filter (forward, then backward) *without* padding, so edge transients are visible; the result is identical to two calls of `sosfilt`. Corner 4 means a 4th-order design, so the effective roll-off is 48 dB per octave per edge.
- **Live page** (`core/live.py`). SciPy's `butter(2, [f_min, min(f_max, 0.45 fs)])` applied with the padded `sosfiltfilt` to each contiguous chunk of the ring buffer, after removing the chunk's mean. The upper corner is limited to $0.45 f_s = 45$ Hz to stay below the Nyquist frequency, and a chunk shorter than the padding length is only demeaned and not filtered.

Both give a zero-phase band-pass; they differ in order and in edge treatment. The numbers a filtered Live trace shows therefore differ slightly from the same interval exported from the Data page.

4.6 Summary

Key idea

A Butterworth band-pass applied forward and backward is a zero-phase filter whose corner gain is -6 dB, whose roll-off is twice that of one pass, and which is *not* causal. Use it to look at waveforms; use a causal filter to detect them.

5 Spectral analysis

The **Spectra** mode of the Helicorder detail view and the third panel of the Waveform Clipboard show an *amplitude spectrum*: how large the oscillation is at each frequency in a window of data. This chapter derives what that number means and how the application computes it.

5.1 The discrete Fourier transform

For N samples $x[0], \dots, x[N-1]$ the discrete Fourier transform (DFT) is

$$X[k] = \sum_{n=0}^{N-1} x[n]\, e^{-\mathrm{j}2\pi kn/N}, \qquad k = 0, \dots, N-1. \tag{5.1}$$

Bin k corresponds to frequency $f_k = k f_\mathrm{s}/N$. For real data the bins above $k = N/2$ mirror those below, so only $k = 0, \dots, N/2$ (frequencies from 0 to f_N) are kept; this is the *one-sided* spectrum. The spacing of the bins,

$$\Delta f = \frac{f_\mathrm{s}}{N} = \frac{1}{T}, \tag{5.2}$$

is the *frequency resolution*: observing for T seconds cannot separate two frequencies closer than $1/T$. The practical meaning is stated by the rule *longer observation, finer spectrum*.

5.2 Windowing and spectral leakage

Computing a DFT of a finite record is the same as multiplying an infinitely long signal by a rectangular window of length N. Multiplication in time is convolution in frequency, so the spectrum of the signal is convolved with the spectrum of the window: a narrow *main lobe* and a series of *sidelobes*. A pure tone that does not fall exactly on a bin is smeared into neighbouring bins, and the sidelobes of a strong tone can bury a weak one. This is *spectral leakage*.

A *taper* that smoothly reduces the data to zero at both ends lowers the sidelobes, at the price of a wider main lobe. The Hann window

$$w[n] = \tfrac{1}{2}\Big(1 - \cos\frac{2\pi n}{N-1}\Big), \qquad n = 0, \dots, N-1, \tag{5.3}$$

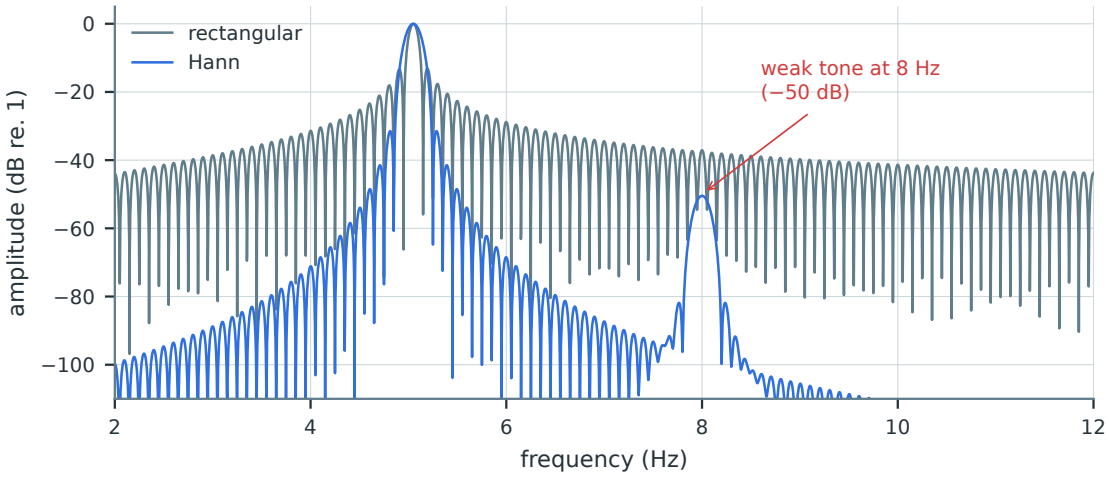


Figure 5.1: Leakage. Spectrum of a tone at 5.05 Hz (not on a bin) plus a tone 50 dB weaker at 8 Hz, $N = 1000$ samples at 100 Hz. The Hann window (blue) reveals the weak tone that the rectangular window (grey) hides.

Table 5.1: Properties of the windows used in RS3D. $\sum w/N$ is the coherent gain; ENBW (equivalent noise bandwidth) and the null-to-null main-lobe width are in bins; the peak sidelobe is relative to the peak.

Window	$\sum w/N$	ENBW	Main lobe	Sidelobe (dB)	Used for
Rectangular	1.000	1.000	2.0	−13.3	(no taper)
Hann	0.500	1.501	4.0	−31.5	amplitude spectrum
Tukey, $\alpha = 0.25$	0.875	1.102	2.3	−13.6	spectrogram

is the common compromise (Harris 1978). Figure 5.1 shows a strong tone at 5.05 Hz and a tone 50 dB weaker at 8 Hz: with the rectangular window the weak tone is invisible in the leakage, with the Hann window it is clearly separated.

Table 5.1 gives the measured properties of the three windows used by the application. They were computed numerically for $N = 1024$ with a zero-padded transform.

5.3 Scaling the amplitude spectrum

Raw DFT values grow with N and are changed by the window, so they are not directly interpretable. The application scales the spectrum so that a sinusoid of amplitude A reads A. For $x[n] = A\cos(2\pi f_0 n/f_s)$ with f_0 at the centre of bin k_0, and a window $w[n]$, a short calculation (Appendix A.1) gives

$$|X_w[k_0]| = \frac{A}{2}\sum_n w[n] \quad\Longrightarrow\quad A = \frac{2\,|X_w[k_0]|}{\sum_n w[n]}. \tag{5.4}$$

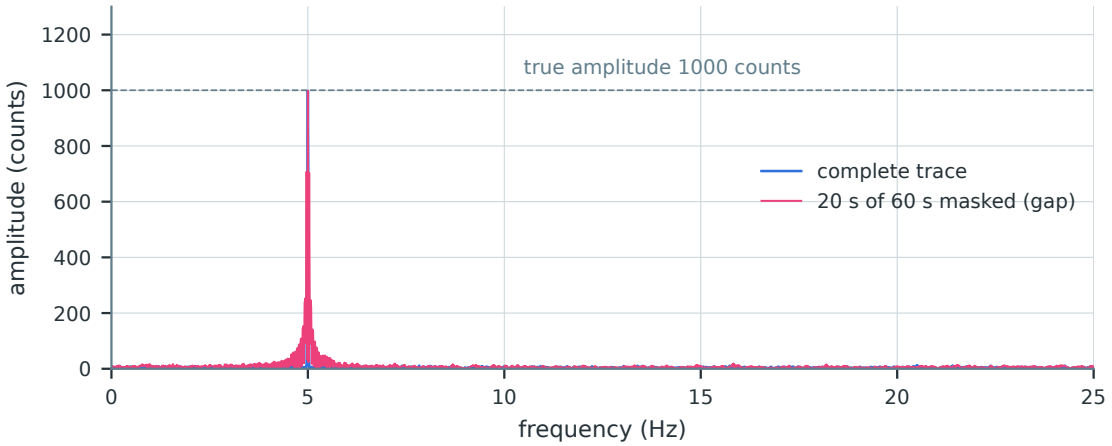


Figure 5.2: Output of the application's `amplitude_spectrum` for a 5 Hz sine of amplitude 1000 counts in noise, with the trace complete (blue) and with 20 s of 60 s masked (pink). Both peak at 1000 counts.

The amplitude spectrum is therefore

$$A[k] = \frac{2\,|X_w[k]|}{\sum_n w[n]}, \tag{5.5}$$

which is in the units of the signal (counts, or m/s). The factor 2 accounts for the energy that the one-sided spectrum folds in from negative frequencies. Equation (5.4) assumes the tone lies on a bin centre. A tone that falls midway between two bins reads low by the window's *scalloping loss*, about 1.4 dB (15 %) for the Hann window.

In RS3D

`amplitude_spectrum` (`core/spectro.py`) computes exactly (5.5) with the **Hann** window (5.3): it subtracts the mean of the *valid* samples, multiplies by the window, takes the real FFT, scales by $2/\sum w$, and discards the zero-frequency bin so that a logarithmic frequency axis is possible. A window with fewer than 8 valid samples returns empty arrays.

Gaps. Samples that are masked or not finite are set to zero after the mean is removed, and the scale uses the sum of the window over the *valid* samples only, $\sum_{n\in\text{valid}} w[n]$. With this rule a tone keeps its true amplitude even when part of the window is missing (figure 5.2); without it, the hidden fill values of an integer trace would dominate the spectrum.

5.4 Amplitude spectrum versus power spectral density

An amplitude spectrum suits *deterministic* signals such as tones, whose amplitude is meaningful. For *random* signals such as ground noise the meaningful quantity is the

one-sided power spectral density (PSD), the power per unit frequency. For a window w the periodogram estimate is

$$S_{xx}[k] = \frac{2\,|X_w[k]|^2}{f_s \sum_n w[n]^2} \qquad (\text{counts}^2/\text{Hz}), \tag{5.6}$$

which integrates over frequency to the variance of the signal. The two are related, for a tone, by $S_{xx}\,B_{\text{ENBW}} = A^2/2$, where $B_{\text{ENBW}} = \text{ENBW} \cdot \Delta f$ is the equivalent noise bandwidth in hertz. The Spectra mode shows amplitude; the spectrogram and the noise spectrum (Chapters 6 and 7) show PSD in decibels.

5.5 Variance of a spectrum estimate and why averaging helps

The DFT of a random signal is itself random. For Gaussian noise the periodogram (5.6) at a given frequency is proportional to a χ^2 variable with two degrees of freedom, whose standard deviation equals its mean: the estimate does not become more reliable by making N larger, it only becomes finer. This is the jagged look of a single-window spectrum in figure 5.3.

Welch (1967) reduced the variance by dividing the record into K (possibly overlapping) segments, computing a windowed periodogram of each, and averaging: the variance falls roughly as $1/K$ at the price of resolution K times coarser than the whole record would allow. The smooth blue curve in figure 5.3 is the root-mean-square average of 24 two-minute windows of the same night-time noise.

Key idea

An amplitude spectrum is exact for tones and scaled by $2/\sum w$; a single-window spectrum of noise is always jagged, and only averaging, as the spectrogram and noise spectrum do, makes it reliable.

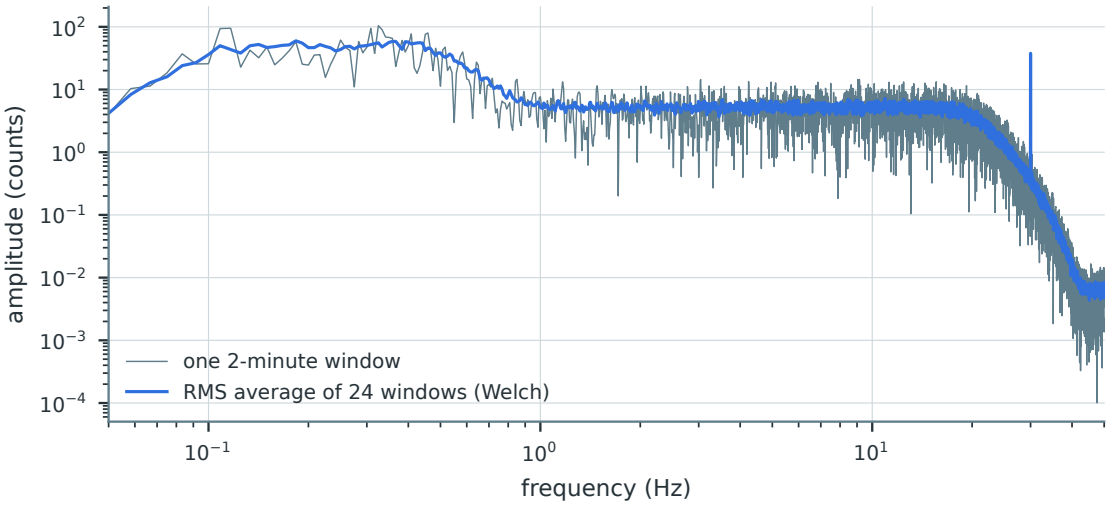


Figure 5.3: Amplitude spectrum of the night-time demo noise (EHZ, from 02:00 UTC). A single two-minute window (grey) is jagged; the RMS average of 24 consecutive windows (blue) is smooth. The narrow line at 30 Hz is a deliberate instrument line in the demo data.

6 The spectrogram

A spectrum describes a whole window. Earthquakes, passing vehicles and instrument faults are *events in time*, and an observer wants to see *when* each frequency was present. The spectrogram answers this by computing a spectrum for many short, overlapping windows and arranging them side by side.

6.1 The short-time Fourier transform

Choose a window of N samples and a *hop* of H samples. The short-time Fourier transform (STFT) is

$$X[m,k] = \sum_{n=0}^{N-1} x[n + mH]\, w[n]\, e^{-\mathrm{j}2\pi kn/N}, \tag{6.1}$$

the DFT of the m-th windowed segment. Each segment is assigned the time of its *centre*, $(mH + N/2)/f_\mathrm{s}$ after the start of the record. The spectrogram is the squared magnitude, converted to a power spectral density by (5.6) and to decibels:

$$P[m,k] = 10 \log_{10} S_{xx}[m,k] \qquad (\text{dB re. 1 count}^2/\text{Hz}). \tag{6.2}$$

The overlap of consecutive windows is $N - H$; the application uses 50 %, so $H = N/2$.

6.2 The time–frequency trade-off

A window of length $T_w = N/f_\mathrm{s}$ seconds has frequency resolution $\Delta f = 1/T_w$ (Eq. 5.2) and time resolution of about T_w. Making one finer makes the other coarser: it is not a limitation of the algorithm but of physics, an instance of Gabor's uncertainty relation $\sigma_t\, \sigma_f \geq 1/4\pi$ (Gabor 1946). Figure 6.1 shows the same 90 s of the local earthquake with three window lengths. A 0.5 s window locates the P and S arrivals sharply in time but cannot resolve frequency finer than 2 Hz; a 10 s window resolves 0.1 Hz but blurs the arrivals over ten seconds.

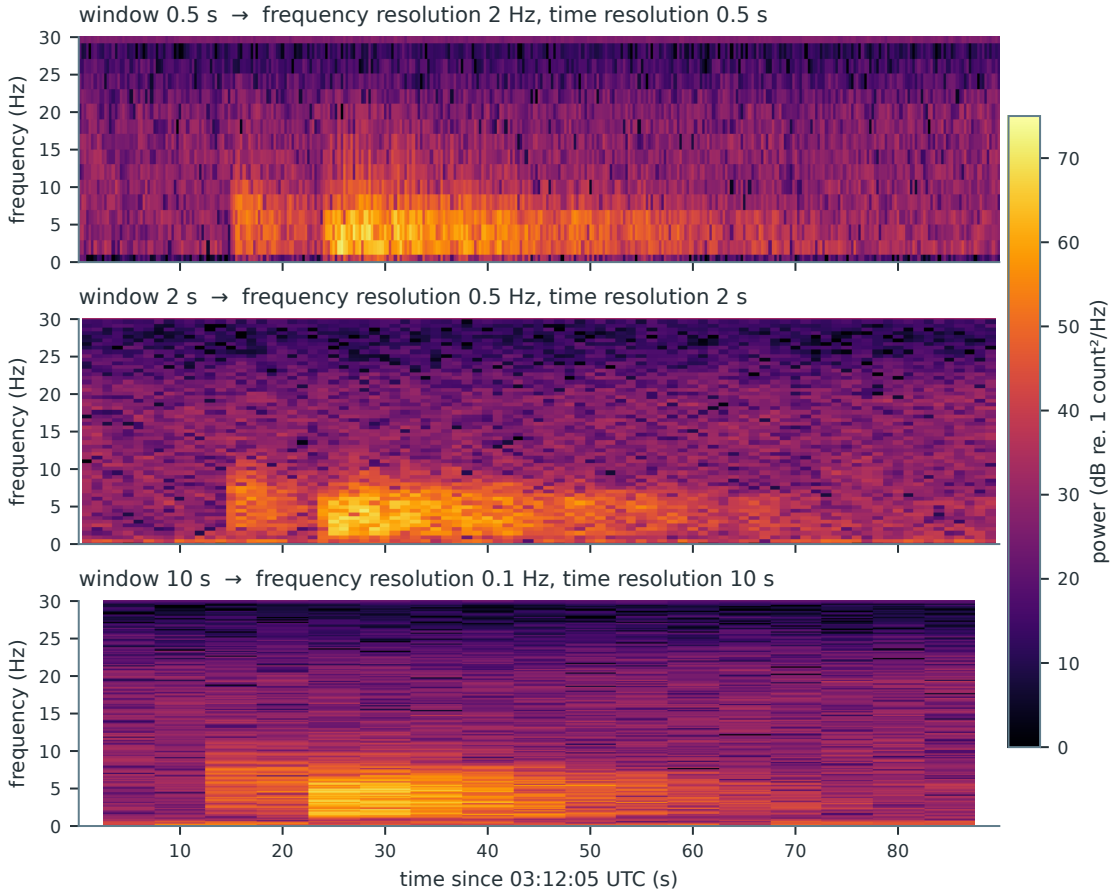


Figure 6.1: The demo local earthquake (EHZ, 03:12:05–03:13:35 UTC) with three window lengths, computed by the application's `spectrogram`. Short windows localise the P and S onsets in time; long windows resolve frequency.

6.3 The window used by RS3D

The application calls SciPy's `spectrogram` without specifying a window, so SciPy's default applies: a periodic **Tukey window with** $\alpha = 0.25$, which is flat in the middle and tapers with half-cosine edges over 25 % of the window. Table 5.1 gives its measured properties: it is close to rectangular, with coherent gain 0.875, noise bandwidth 1.10 bins, and a peak sidelobe only −13.6 dB below the peak. This is a defensible choice for *locating* events (the main lobe is narrow and little signal is lost) but it leaks more than the Hann window of the amplitude spectrum.

Leakage from strong low-frequency noise

> Because the Tukey window has high sidelobes that decay slowly, a very strong low-frequency signal, such as the microseism, leaks a little energy into higher

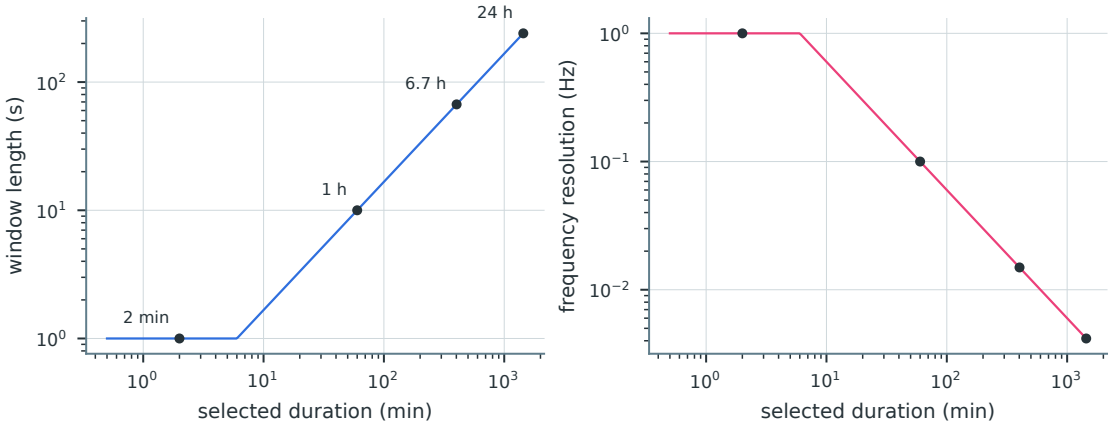


Figure 6.2: The window-length rule (6.3) (left) and the frequency resolution it implies (right) as a function of the selected duration.

Table 6.1: Window length, frequency resolution and number of columns for typical selections.

Selection	T_w (s)	Δf (Hz)	columns
2 minutes (detail view)	1	1.0	239
1 hour	10	0.1	719
6.7 hours	67	0.015	719
24 hours	240	0.004	719

bins. For a typical RS3D day the effect is smaller than the dynamic range of the colour scale, but a spectrogram of data with a large low-frequency component should be read with this in mind. Replacing the window by a Hann window is a one-line change should it ever be needed.

6.4 Choosing the window length: the 360 rule

No single window length suits both a two-minute detail view and a 24-hour overview. The application chooses it from the duration T of the selection:

$$T_w = \min(\max(T/360,\ 1\ \mathrm{s}),\ 600\ \mathrm{s}). \tag{6.3}$$

With 50 % overlap a record of duration T yields about $2T/T_w - 1 \approx 719$ columns, a few hundred to a thousand, which is about what a screen can display and what the browser can plot quickly, whatever the duration. The limits keep the window between 1 s (so that even a short selection has a usable number of samples) and 10 minutes. Figure 6.2 shows the resulting window length and frequency resolution; table 6.1 gives worked examples.

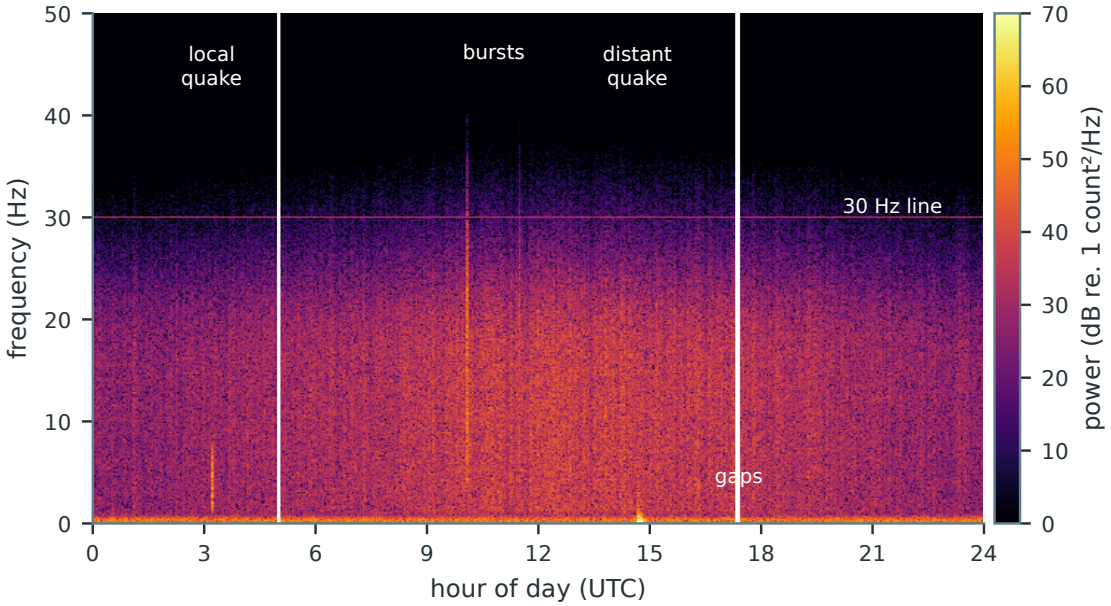


Figure 6.3: A full-day spectrogram of the demo EHZ channel computed with the application's functions (window 240 s). The day–night change in cultural noise, the local and distant earthquakes, two noise bursts, the 30 Hz instrument line and the two gaps (white) are all visible.

6.5 Gaps

A window that touches a gap would mix real signal with filler. The application therefore sets the samples of a gap to zero for the transform and then marks *every column whose window overlaps a gap* as not-a-number, drawn as blank. The columns on either side of a gap are computed from complete windows only, so a gap does not smear into its neighbours (figure 6.3, where two gaps are visible as white bars).

6.6 From numbers to pixels

The application never sends the full grid to the browser. The (frequency × time) matrix is *binned* to at most 400 × 200 cells; each cell shows the mean of the decibel values it covers, and cells with no valid value are left out. Averaging *decibels* is the same as taking a geometric mean of power, which slightly underestimates narrow peaks compared with averaging power. The colour scale runs from the 2nd to the 98th percentile of the displayed values, so that a few extreme cells do not wash out the picture.

6.7 Reading a spectrogram

Earthquake a vertical streak starting abruptly at the P arrival; energy at higher frequencies arrives first and the S wave follows, with a coda fading at lower frequencies.

Man-made burst
a short, broadband streak with no coda; compare the two bursts at about 10–11 h in figure 6.3.

Narrow horizontal line
a persistent single frequency: usually electrical interference or machinery, here the 30 Hz instrument line.

Slow day–night change
cultural noise, strongest in working hours.

Low-frequency band
the ocean microseism, near 0.1–0.3 Hz.

In RS3D

`spectrogram(trace, window_s, overlap=0.5)` in `core/spectro.py` returns window-centre times, frequencies and decibel values; `window_for(duration)` is Eq. (6.3). The page code (`app/charts.py`, `spectrogram_options`) performs the binning and colour scaling.

7 The noise spectrum

The **Noise spectrum** panel on the Data page summarises how much energy the ground contained at each frequency during the selected time, as a median curve with a band around it. It is the quickest way to judge a site, to compare day with night, and to spot a new source of interference.

7.1 Definition

Start from the spectrogram grid $P[m, k]$ of Eq. (6.2) (columns m are times, rows k frequencies).

1. Drop every column that is entirely not-a-number, that is, every window touching a gap.
2. Drop the zero-frequency row.
3. For each frequency f_k compute three *percentiles* across the remaining columns:

$$P_{10}[k], \qquad P_{50}[k] = \text{median}_m\, P[m, k], \qquad P_{90}[k].$$

The panel draws P_{50} as a line and the range P_{10}–P_{90} as a band on a logarithmic frequency axis.

Percentiles, and not the mean, are used because they are *robust*: an earthquake or a noise burst occupies a small fraction of the columns, so it moves P_{90} but leaves the median essentially at the level of the background noise. Figure 7.1 compares one night hour with one midday hour of the demo day.

7.2 Relation to the McNamara–Buland PPSD

The standard tool for describing the noise of a seismic station is the *probabilistic power spectral density* (PPSD) of McNamara and Buland (2004). Its recipe is: cut the data into one-hour segments; estimate a PSD for each with Welch's method, correct it for the instrument response; smooth it into narrow period bands; and accumulate, for every period, a histogram of the resulting power levels. The histograms, together with the global new low-noise and high-noise models of Peterson (1993), show at a glance how quiet and how variable a site is.

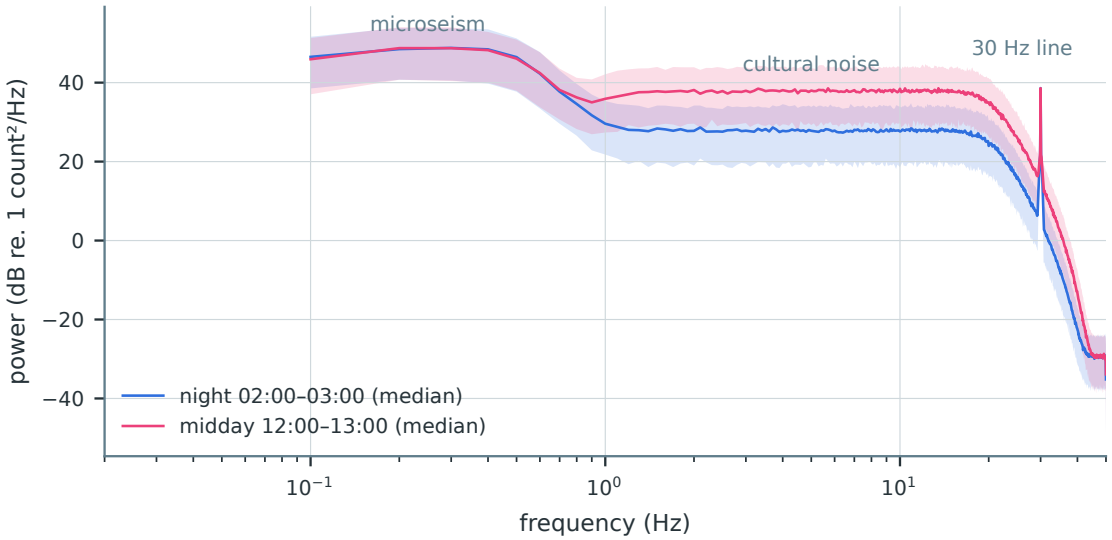


Figure 7.1: Noise spectrum of the demo EHZ channel for 02:00–03:00 (blue) and 12:00–13:00 (pink): median and 10th–90th percentile band, from the application's `spectrogram` and `psd_summary`. Cultural noise above 1 Hz is clearly stronger at midday.

Table 7.1: RS3D noise spectrum versus the PPSD of McNamara and Buland (2004).

	PPSD	RS3D noise spectrum
Segments	one-hour, overlapping	the spectrogram windows of the chosen range
Averaging inside a segment	Welch average of sub-segments	none: each window is one periodogram
Instrument response	removed	**not** removed (counts^2/Hz)
Frequency smoothing	narrow octave fractions	none (linear bins)
Summary	full histogram per period	10th, 50th and 90th percentile
Reference models	NLNM and NHNM	none

The noise spectrum of RS3D is a deliberately simplified relative of the PPSD, and the differences matter when the two are compared.

Not comparable with the Peterson noise models

Because the response is not removed, the RS3D curve is in counts^2/Hz, not in the acceleration units of the Peterson models, and its shape includes the geophone's own roll-off below 4.5 Hz. It must be used for *relative* comparisons on the same instrument (day versus night, this week versus last week), not to rank a site against published noise levels.

7.3 What the curves tell you

Microseism, 0.1–0.3 Hz
: ocean waves coupling to the sea floor; present everywhere, strongest in winter storms. A short-period geophone sees it faintly because its response falls steeply below 4.5 Hz.

Above 1 Hz
: cultural noise (traffic, machinery, people). It changes with the time of day and the day of the week; the difference between the two curves of figure 7.1 is a direct measure of it.

Narrow peaks
: a single frequency that never changes: interference. The 30 Hz line of the demo data is an example.

Roll-off near 50 Hz
: the digitiser's anti-alias filter (Chapter 2); it is a property of the instrument, not of the ground.

In RS3D

`psd_summary(freqs, power_db)` in `core/spectro.py` returns $(f, P_{10}, P_{50}, P_{90})$ using `numpy.percentile` on the decibel values, after dropping gap columns and the zero-frequency bin. The chart is `psd_options` in `app/charts.py`.

8 Data quality: gaps and completeness

A seismograph is only useful while it is recording. The application measures and displays three aspects of *data availability*: what is on the laptop and what is still on the Shake (the coverage bar), what is missing inside the selected window (Data health), and whether the Shake's clock can be trusted (Chapter 2).

8.1 Samples, spans and gaps

A trace is a set of samples at times $t_i = t_0 + i/f_s$. Records and traces end where a gap begins, so the whole day is a list of *spans* $[s_j, e_j)$, where e_j is the time just after the last sample of the span, $e_j = s_j + N_j/f_s$. Two spans are *contiguous* when the hole between them is at most half a sample, which absorbs timing rounding; contiguous spans are merged. What counts as a gap depends on who is asking, so the application uses three tolerances.

8.2 Completeness

For a selection $[a, b)$ with $T_{\text{sel}} = b - a$ the application computes, for every channel, the set of times with data (the union of its clipped spans) and from it the set of times *missing* on that channel. The *gaps* of the selection are the union, over channels, of the missing sets: a moment counts as a gap if *any* selected channel has no data. The *completeness* is

$$C = 1 - \frac{\sum_g (e_g - s_g)}{T_{\text{sel}}}, \qquad 0 \le C \le 1, \tag{8.1}$$

Table 8.1: Gap tolerances used by different parts of the application.

Use	Tolerance	Rule
Coverage bar (`core/coverage.py`)	$0.5/f_s$	spans closer than half a sample are merged into one
Data health (`core/health.py`)	$0.5/f_s$	same rule, applied per channel
Helicorder rows (`core/helicorder.py`)	$1.5/f_s$	a break is drawn only when the hole exceeds one and a half sample intervals

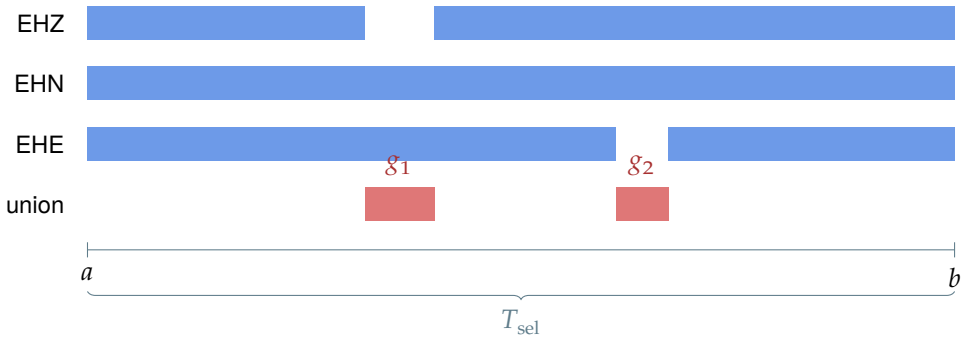


Figure 8.1: Completeness of a selection. Each channel has its own holes; the gaps of the selection (g_1, g_2) are the union of all of them, and $C = 1 - (|g_1| + |g_2|)/T_{\text{sel}}$. In this sketch $C = 1 - (0.8 + 0.6)/10 = 86\,\%$.

where the sum runs over the gaps g of the union. If no channel has any data in the selection there is one gap equal to the whole selection and $C = 0$. Figure 8.1 illustrates the construction.

In RS3D

`health(stream, start, end)` in `core/health.py` returns the completeness and the list of gaps, each as a pair of times; the Data page prints it as a percentage and lists up to 50 gaps with their durations in the chosen time zone. The leading and trailing holes of a selection (data starting after a or ending before b) are gaps like any other.

8.3 Coverage: laptop, Shake, or nothing

The coverage bar above the Data page's preview classifies every instant of a view into three states: *local* (the samples are on the laptop), *remote* (the day exists on the Shake but is not downloaded) and *gap* (no data anywhere). The rule, evaluated per segment between cut points at midnight and at span edges, is:

1. if the midpoint of the segment lies inside a local span, the state is local;
2. otherwise, if that day exists on the Shake *and* the laptop does not have the day file, or the day is the newest day on the Shake (still being written), the state is remote;
3. otherwise it is a gap.

The "newest day" exception is why today's file is always offered for download again: the Shake appends to it continuously, so a copy made earlier is incomplete.

8.4 Why a gap is never filled

Interpolating across a gap would be easy and would make plots prettier, but it would manufacture data. RS3D keeps gaps as gaps in every stage: separate traces on loading, separate filter runs, `NaN` separators in the plotted lines, blank spectrogram columns, and an explicit completeness figure. When a gap is shorter than the window of an operation, that operation excludes the window rather than guessing its content.

9 Visualisation algorithms

One day of one channel is $86400 \times 100 = 8.64$ million samples; a screen is a few thousand pixels wide. Every plot of RS3D therefore *reduces* the data before drawing it. How it is reduced decides whether a short, important spike survives. This chapter describes the reductions and the geometry of the helicorder.

9.1 Reducing a signal for display

9.1.1 Stride decimation and why it can lose a spike

The simplest reduction keeps every q-th sample. It is fast, and for a signal that is oversampled relative to its bandwidth it is harmless, but for a spiky signal it is wrong: the sampling theorem (Chapter 2) applies again, and a feature narrower than q samples is either kept or discarded depending on where it falls in the grid. Figure 9.1 (middle) shows a one-sample spike that stride decimation misses.

9.1.2 Min/max decimation

A seismogram plot is read mainly through its *envelope*, the extremes within each short interval. Min/max decimation preserves exactly that. The record is divided into B consecutive bins; from each bin the algorithm keeps the sample with the smallest value and the sample with the largest value (one point if they coincide), in time order. The plot then has at most $2B$ points and contains the true extremes of every bin:

$$(t_b^{\min}, x_b^{\min}),\ (t_b^{\max}, x_b^{\max}), \qquad x_b^{\min} = \min_{n \in b} x[n], \quad x_b^{\max} = \max_{n \in b} x[n]. \tag{9.1}$$

Peak amplitudes and their times are therefore *exact*; what is lost is the detailed shape inside a bin, which is below the pixel resolution anyway. The cost is linear in the number of samples.

Gaps. The input to the decimator marks gaps with `NaN` samples. The record is first split at these markers, each segment is decimated on its own with a share of the point budget proportional to its length (at least one bin), and a `NaN` is re-inserted between segments, so the plotted line breaks at every gap and no line is drawn across missing data.

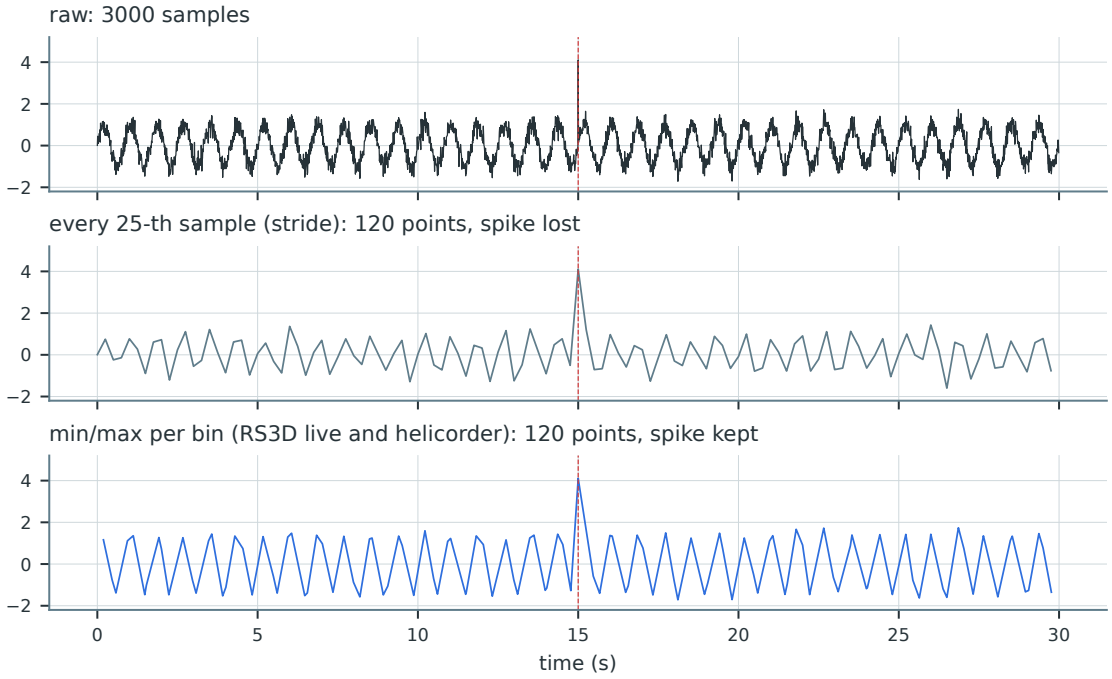


Figure 9.1: Reducing 3000 samples to 120 points. Top: the raw signal with one single-sample spike (dashed line). Middle: every 25th sample, which misses the spike. Bottom: min/max decimation as implemented in `minmax_decimate`, which keeps it.

In RS3D

`minmax_decimate(t, v, max_points)` in `core/live.py` is used by the Live view (up to 1500 points per channel) and by the helicorder (3000 points per row, `POINTS_PER_ROW`). **The waveform preview on the Data page and the detail view are different:** `set_stream_data` in `app/charts.py` thins each trace with a stride of $\lceil N/4000 \rceil$ and then lets the charting library apply its own "largest-triangle-three-buckets" downsampling (Steinarsson 2013), which also preserves visual shape but does not guarantee exact extremes. For the short windows of the detail view (a few thousand samples) no reduction occurs at all, so the issue arises only for long previews. Moving the preview to min/max decimation would be a straightforward improvement.

9.2 The helicorder

A helicorder (from *helical recorder*) wraps a long record into rows, like lines of text, so that a day fits on one page and an event stands out as a burst on one line.

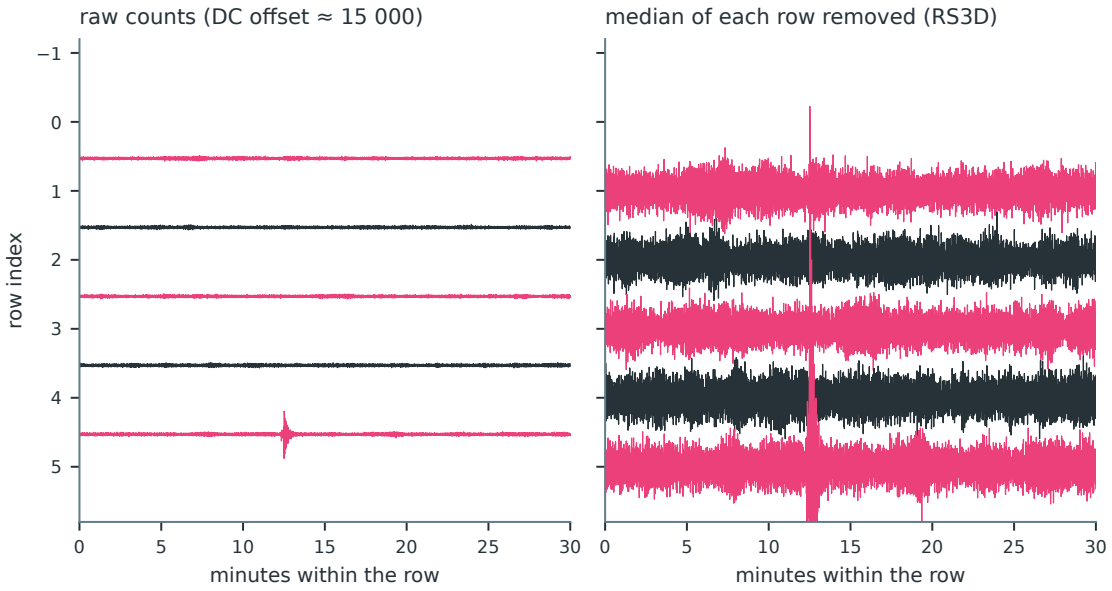


Figure 9.2: Six rows of the demo EHZ channel with an offset of about 15 000 counts. Left: raw counts: the offset dominates the scale and every trace is a nearly flat line shifted by half a row. Right: the median of each row removed: the signal fills the row height and the small local earthquake is visible in the lower rows.

9.2.1 Geometry

Let t_0 be midnight UTC, $\Delta = 1800$ s the row length (30 minutes) and $R = 48$ the number of rows. Row i shows the interval $[t_0 + i\Delta,\ t_0 + (i+1)\Delta)$. A sample at time t in row i is drawn at horizontal position

$$x = \frac{t - (t_0 + i\Delta)}{60} \quad \text{minutes}, \qquad 0 \le x < 30, \tag{9.2}$$

and vertical position

$$y = i - \frac{v(t) - m_i}{S}, \tag{9.3}$$

with the y axis pointing down, so a positive excursion moves the trace up. Here m_i is the *median* of row i and S is a single amplitude scale shared by all rows.

9.2.2 Removing the offset

Raw counts sit on a DC offset (Chapter 3). Without m_i the offset divided by S would shift each row by a fixed fraction of a row, and, worse, would enter the scale S (below), flattening the signal. Figure 9.2 contrasts the two. The median is used because, unlike the mean, it is not pulled by an earthquake inside the row.

9.2.3 The amplitude scale

The scale S decides how tall the traces are. It must be large enough that adjacent rows do not collide and small enough that quiet signals are visible. The application sets

$$S = \frac{2\,P_{99}(\,|v_{\mathrm{dec}} - m|\,)}{F}, \tag{9.4}$$

where P_{99} is the 99th percentile of the absolute values of all plotted (decimated, median-removed) points of the day, and F is the user's **Scale** factor ($0.5\times$ to $8\times$). With $F = 1$, the 99th percentile maps to half a row, so ordinary signal fills about one row and only the strongest 1 % exceed it. A percentile is used rather than the maximum so that one huge event does not squash a whole quiet day into flat lines. Note that because the points are min/max extrema, P_{99} of the decimated data is somewhat larger than that of the raw samples.

9.2.4 From a click to a time

The helicorder page must convert a mouse click into a time. ECharts reports clicks on *data points*, not on empty space, so the page lays an invisible grid of 48×180 points over the plot: one every 10 seconds along each row, drawn as transparent 10×14 pixel rectangles. A click reports the nearest grid point as the pair $(x_{\min}, i)$, and the time is

$$t = t_0 + \Delta\left(i + \frac{x_{\min}}{30}\right), \qquad x_{\min} \text{ clamped to } [0, 30]. \tag{9.5}$$

Equation (9.5) (`click_to_time` in `core/helicorder.py`) is exact for the grid point; the point itself is at most 5 s from where the pointer was. The chart height is fixed to $48 \times 14 + 44$ pixels so that each row is exactly 14 pixels and a pixel row corresponds to a row index without rounding.

9.3 The detail view

A click opens a window of $T_w = 120$ s centred on the clicked time. The toolbar changes this window with a small set of pure functions (`core/helicorder.py`):

Zoom out / in multiplies the span by 2 or $\frac{1}{2}$ about the centre, clamped to between 5 and 3600 seconds.

Earlier / later shifts the window by $\mp\frac{1}{2}$ of its span.

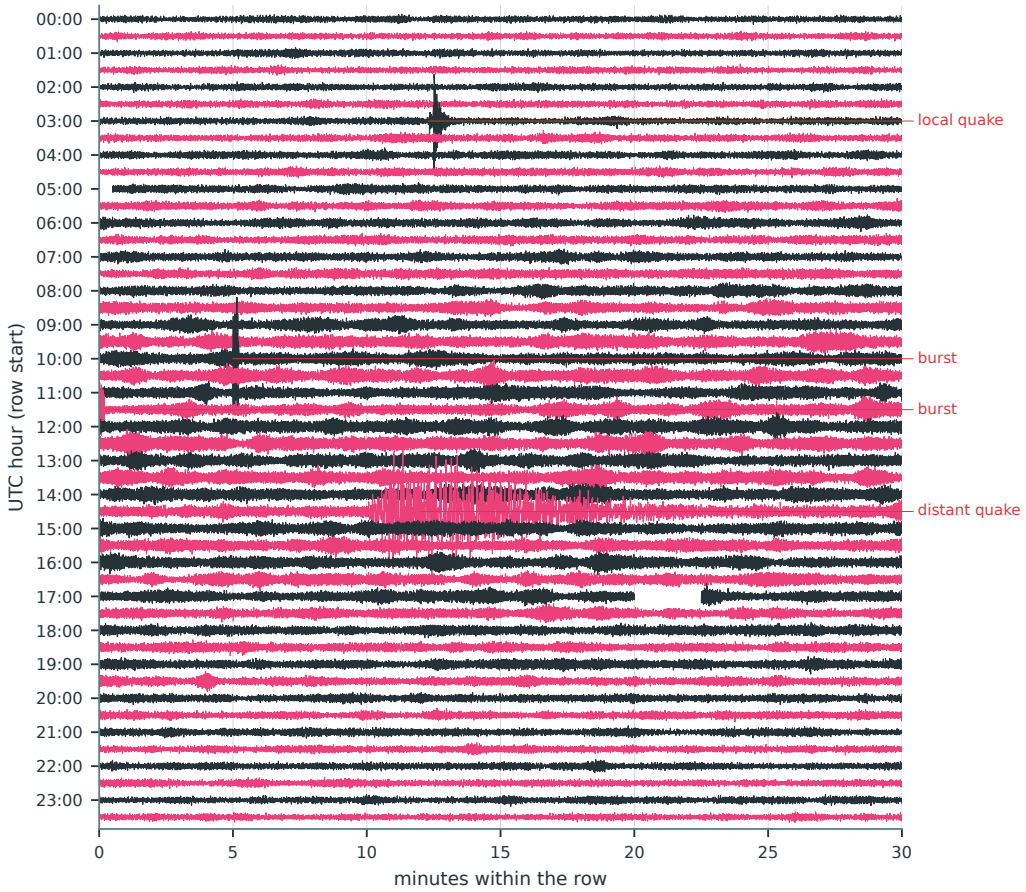


Figure 9.3: A day of the demo EHZ channel drawn with the application's `build_rows` and `scale_for`. Rows are 30 minutes; the day–night change, the local earthquake (03:00 row), two noise bursts, the distant earthquake (14:30 row) and a gap at 17:20 are visible.

Box zoom a dragged rectangle gives a time interval $[x_0, x_1]$ (epoch ms) and an amplitude interval. The new window is the rectangle's time span clipped to the current window and widened, centred, to at least 5 s; a rectangle that does not overlap the window changes nothing. The amplitude interval, rounded to four significant figures for readable axis labels, fixes the vertical axis.

Fit clears any amplitude zoom, so the axis rescales to the data in view; the time window is kept.

Key idea

Min/max decimation keeps peaks exact; the helicorder removes each row's median and scales all rows by the same robust percentile; a click becomes a time through a 10-second grid.

10 Event detection and alerts

An observer cannot watch a screen all day. The purpose of an *alert* is to say, without a human present, that something happened: the ground moved more than it normally does. This chapter explains the standard way of deciding that, the STA/LTA trigger (Allen 1978), shows how it behaves on data whose true events are known, and describes how RS3D's alert pipeline is designed around it.

Status of this chapter

> Sections 10.1 to 10.7 describe standard theory and measurements made with the application's demo data and ObsPy's reference implementation. Section 10.9 is a *design proposal*: the alert feature of RS3D is not yet implemented, and the proposal will be revised after it is.

10.1 The detection problem

Detection is a decision between two hypotheses at every instant: H_0, *only background noise*, and H_1, *noise plus a seismic signal*. Any decision rule makes two kinds of mistakes: a *false alarm* (declaring H_1 when H_0 is true) and a *missed detection* (declaring H_0 when H_1 is true). Lowering the threshold at which an event is declared catches weaker events but raises the false-alarm rate, and vice versa; no rule escapes the trade-off, and the art lies in choosing a rule whose false alarms are rare enough to tolerate at an acceptable sensitivity. For an alert that wakes somebody at night the tolerable false-alarm rate is very low.

The difficulty is that seismic noise is not a fixed distribution. It changes with the time of day, the weather and the neighbours' habits. A detector must therefore compare the *present* signal with the *recent* noise level, not with a constant. This is the idea of the STA/LTA trigger.

10.2 The characteristic function

Before averaging, the signal is turned into a non-negative *characteristic function* (CF) $c[n]$ that grows when a signal is present. The simplest choices are the squared amplitude, $c[n] = x[n]^2$ (an instantaneous power), or the absolute value $|x[n]|$. Allen's original function also includes the squared first difference, which responds to abrupt changes. ObsPy's reference implementation, and RS3D's design, use $c[n] = x[n]^2$.

Two preprocessing steps are essential before the CF is formed.

1. **Remove the offset.** The DC component of raw counts (Chapter 3) would be squared into a huge constant that masks any event. A band-pass filter that rejects frequencies below the lower corner removes it.
2. **Band-pass filter, causally.** Restricting the signal to the band where the events of interest have their energy raises the signal-to-noise ratio. For the filter to be usable in real time and not to blur the onset (Chapter 4) it must be *causal*.

10.3 The classic STA/LTA

Let N_s and N_l be the lengths, in samples, of a short and a long window, $N_s \ll N_l$. The short-term average (STA) and long-term average (LTA) of the CF at sample i are the means over the two *trailing* windows:

$$\mathrm{STA}_i = \frac{1}{N_s} \sum_{j=i-N_s+1}^{i} c[j], \qquad \mathrm{LTA}_i = \frac{1}{N_l} \sum_{j=i-N_l+1}^{i} c[j], \qquad R_i = \frac{\mathrm{STA}_i}{\mathrm{LTA}_i}. \tag{10.1}$$

The STA follows the signal quickly and estimates the *current* power; the LTA changes slowly and estimates the *background*. In quiet noise $R \approx 1$. When an event arrives the STA rises almost at once while the LTA, which averages over a long past, hardly moves, so R jumps. The detector declares an event when R crosses a *trigger-on threshold* r_{on} upward and ends it when R falls below a lower *trigger-off threshold* $r_{\mathrm{off}} < r_{\mathrm{on}}$. Using two thresholds, *hysteresis*, prevents rapid on–off flicker when R hovers near one value.

10.3.1 The warm-up period

LTA_i is meaningless until N_l samples have been seen. ObsPy sets the ratio to zero for the first samples; a live monitor must not raise an alert during this period, which is N_l/f_s seconds long (30 s in the example).

10.4 The recursive STA/LTA

The classic form needs the last N_l samples in memory. A *recursive* variant replaces each moving average by an exponential one:

$$\mathrm{STA}_i = \mathrm{STA}_{i-1} + \frac{c[i] - \mathrm{STA}_{i-1}}{N_s}, \qquad \mathrm{LTA}_i = \mathrm{LTA}_{i-1} + \frac{c[i] - \mathrm{LTA}_{i-1}}{N_l}. \tag{10.2}$$

Unrolling the recursion shows that sample $c[i-k]$ carries the weight $\frac{1}{N}(1-\frac{1}{N})^k$: a geometric, not a rectangular, window, with a *time constant* of N samples ($\tau_s = N_s/f_s$,

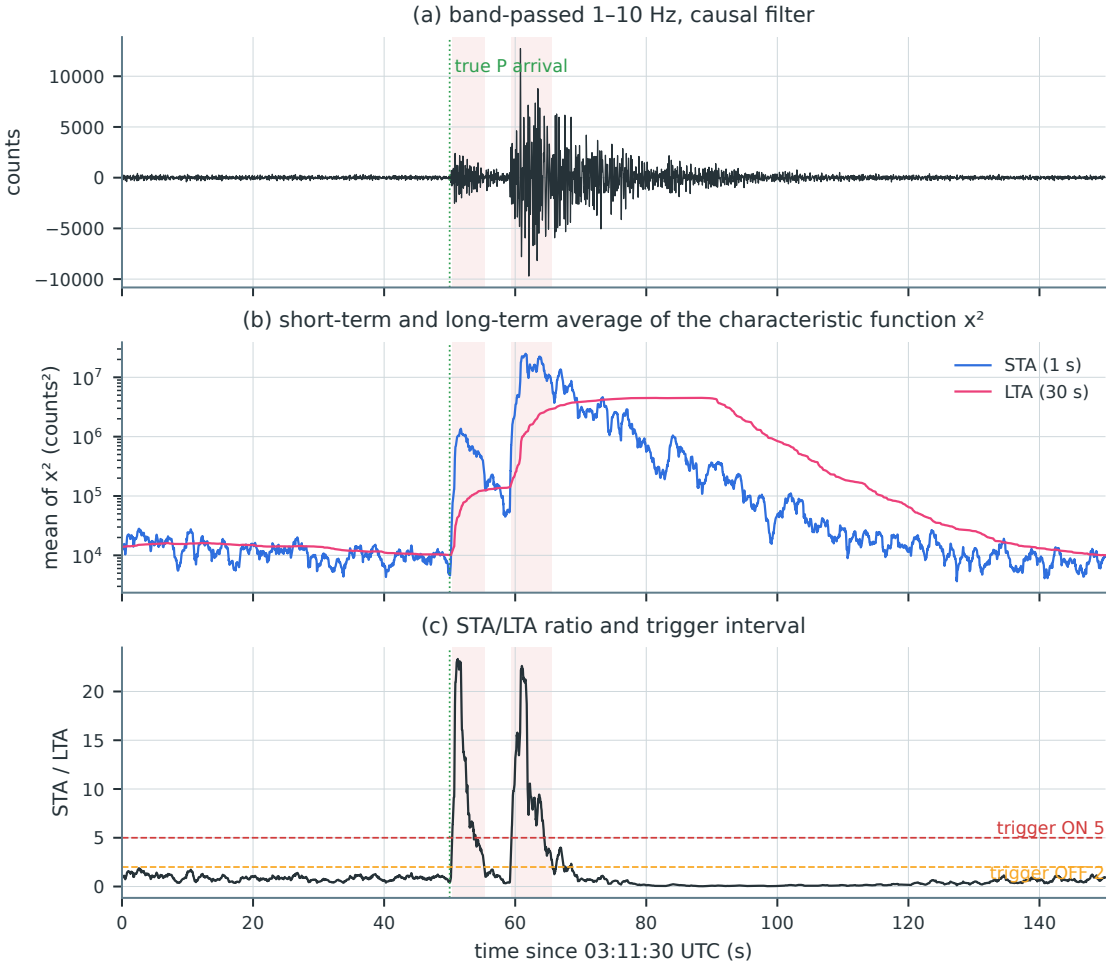


Figure 10.1: STA/LTA on the demo local earthquake (EHZ). (a) The signal band-passed 1–10 Hz with a causal filter. (b) STA (1 s) and LTA (30 s) of the characteristic function x^2: the STA jumps by orders of magnitude at the P arrival while the LTA lags. (c) The ratio, the thresholds ($r_{\text{on}} = 5$, $r_{\text{off}} = 2$) and the trigger intervals (pink). The ratio falls back below the threshold between the P and the S wave, so the single earthquake produces two trigger intervals.

$\tau_l = N_l/f_s$ seconds) (Withers et al. 1998; Trnkoczy 2012). The recursion needs only two numbers of state and one multiplication per sample, independent of N_l, which makes it the natural choice for a monitor that runs continuously on a stream. For the same N_s and N_l the two forms give similar but not identical ratios.

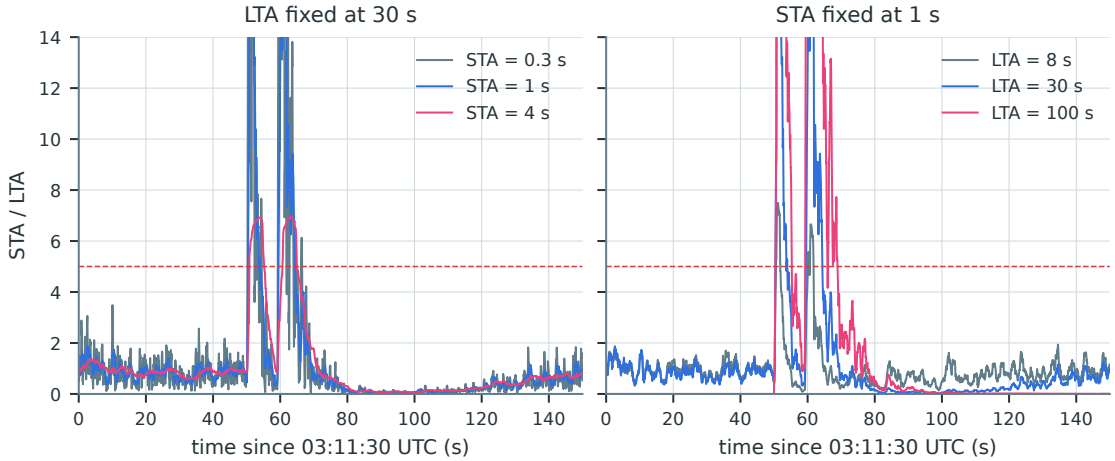


Figure 10.2: Effect of the window lengths on the ratio for the same earthquake. Left: STA of 0.3, 1 and 4 s with LTA 30 s. A longer STA gives a lower, smoother peak. Right: LTA of 8, 30 and 100 s with STA 1 s. A short LTA is contaminated by the event itself and the ratio collapses quickly. Dashed: threshold 5.

10.5 Setting the parameters

The three groups of parameters have different roles; Trnkoczy (2012) gives a systematic treatment.

STA length should be a few periods of the dominant signal frequency: long enough that the STA is not driven by a single large sample, short enough to react quickly. A short STA finds short, impulsive events; a long STA favours emergent ones. Figure 10.2 (left) shows the effect.

LTA length should be much longer than the STA (a factor of 10 or more) so that an event does not contaminate its own background estimate, yet short enough to follow the slow change of the noise. Figure 10.2 (right) shows the effect.

Thresholds r_{on} sets the sensitivity; r_{off}, typically a fraction of r_{on}, sets how long an event is held. The values of the example, $r_{\mathrm{on}} = 5$ and $r_{\mathrm{off}} = 2$, were chosen from the sweep of figure 10.5.

Pass band the band in which the CF is formed; it should contain the energy of the target events and exclude the dominant noise.

Table 10.1: False alarms per day (events merged within 30 s) for stationary Gaussian noise and for the demo day, with the F-distribution tail probability for one independent ratio value. Detector: causal 1–10 Hz band-pass, recursive STA/LTA, STA 1 s, LTA 30 s, $r_{\mathrm{off}} = 0.4\, r_{\mathrm{on}}$.

Threshold	F tail $P(R > r)$	Stationary noise	Demo day
2	8.5e-03	89	498
2.5	6.0e-04	0	240
3	3.5e-05	0	83
3.5	1.7e-06	0	25
4	7.9e-08	0	7
5	1.4e-10	0	0
6	2.3e-13	0	0

10.5.1 Statistical meaning of the ratio

If the signal is Gaussian noise and the CF is x^2, the average of x^2 over a window follows a scaled χ^2 distribution. For noise that occupies a bandwidth B (hertz) a window of T seconds contains about $\nu \approx 2BT$ independent values, so STA and LTA have $\nu_s = 2BT_s$ and $\nu_l = 2BT_l$ degrees of freedom and, for stationary noise, the ratio R follows an F distribution with (ν_s, ν_l) degrees of freedom. For the 1–10 Hz band ($B \approx 9$ Hz) with a 1 s STA and a 30 s LTA this gives $\nu_s = 18$ and $\nu_l = 540$. The probability that a *single* ratio value exceeds a threshold is then tiny (table 10.1, second column): 1.4×10^{-10} at $r = 5$.

The table's last two columns are the important result. For *stationary* noise, false alarms disappear above $r \approx 2.5$, as theory predicts. The demo day, whose noise changes with the time of day and fluctuates from minute to minute, still produces 83 false events per day at $r = 3$ and does not reach zero until $r = 5$. *Real noise is not stationary*, the LTA lags behind every rise in the noise level, and a threshold chosen from stationary theory is far too sensitive.

Key idea

The threshold must be chosen from the noise of *your* site, over days, not from the theory of stationary noise. A threshold that gives zero false alarms on stationary noise may give dozens per day on a real station.

10.6 Behaviour on the demo day

Figure 10.4 applies the detector (causal 1–10 Hz band-pass, recursive STA/LTA, STA 1 s, LTA 30 s, $r_{\mathrm{on}} = 5$, $r_{\mathrm{off}} = 2$, events closer than 30 s merged) to the whole demo day. Table 10.2 lists the events found. All three are real in the sense that something happened; two of them are not earthquakes.

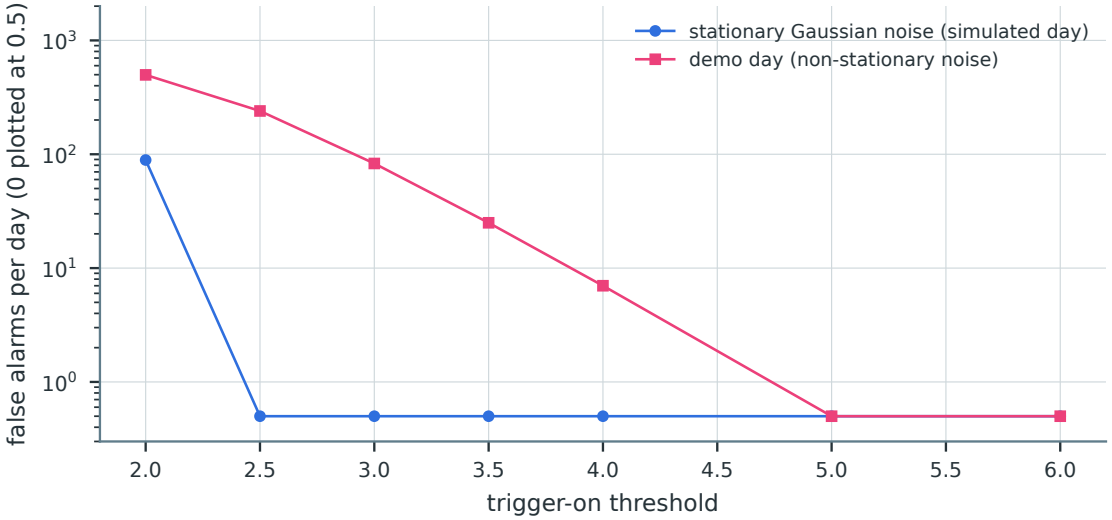


Figure 10.3: False alarms per day against threshold for stationary noise (blue) and the demo day (pink); zeros are plotted at 0.5 on the logarithmic axis.

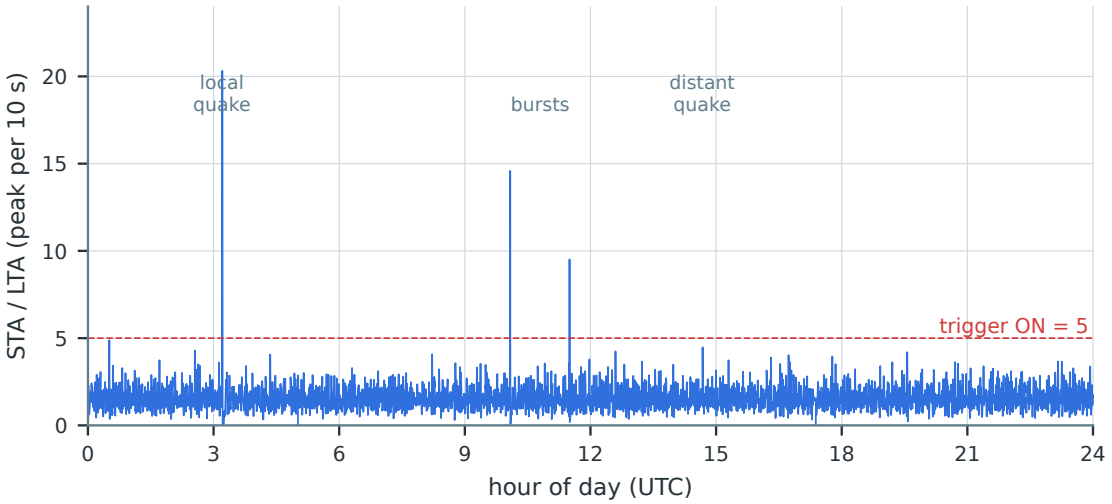


Figure 10.4: The STA/LTA ratio over the demo day (peak per 10 s). Three episodes cross the threshold of 5: the local earthquake and the two noise bursts. The distant earthquake, whose energy is below 1 Hz, stays below it.

Table 10.2: Events detected on the demo day at $r_{\text{on}} = 5$. The "cause" is known because the demo data were constructed.

Onset (UTC)	Duration (s)	Peak ratio	Cause
03:12:20	18	20.3	local earthquake
10:05:00	12	14.6	noise burst
11:30:00	12	9.5	noise burst

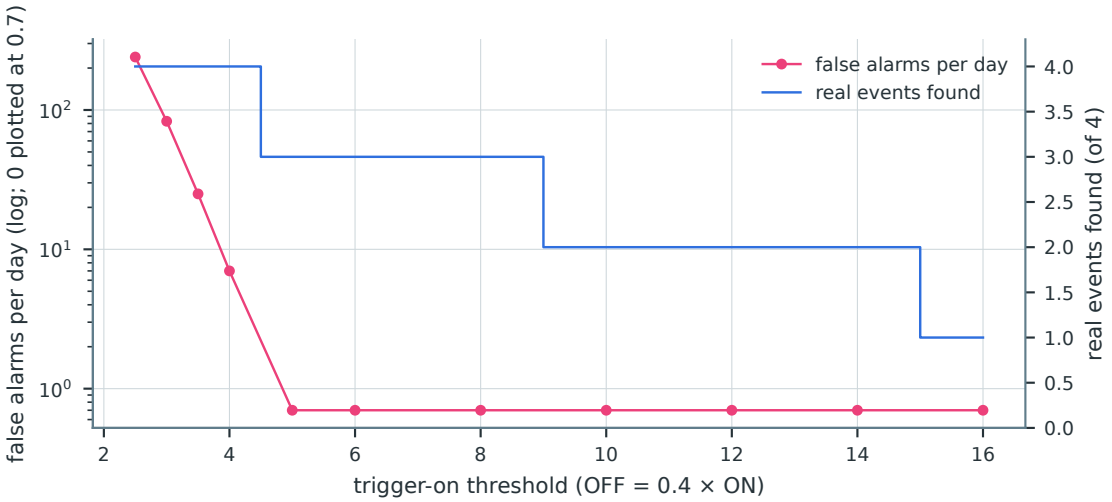


Figure 10.5: Sweep of the trigger-on threshold. False alarms (pink, left axis) vanish at 5; real events found (blue, right axis) fall from 4 to 3 at that threshold, because the distant earthquake is lost, then to 2 and 1 as the two noise bursts are lost one after the other; at the highest thresholds only the local earthquake remains.

10.7 Failure modes and how to reduce them

10.7.1 Man-made noise looks like an earthquake

The two noise bursts of the demo day exceed the threshold by a wide margin: to an energy detector a person stamping next to the sensor and a small local earthquake are both a sudden rise in power. Figure 10.6 shows the difference that can be exploited: the earthquake keeps a long, decaying *coda* and the ratio stays elevated, whereas the burst stops abruptly and the ratio collapses within seconds. Criteria that use this are a minimum duration, the ratio's persistence, and the frequency content (the burst is broadband, the earthquake concentrated in 2–8 Hz).

10.7.2 Other failure modes

Gaps a gap must reset the long average; otherwise the first samples after the gap are compared with a stale background. The detector treats each contiguous stretch separately and observes the warm-up period after each gap.

Offset steps and spikes
a step in the DC level or a single instrument spike produces a short, large CF. The band-pass removes steps; a spike is removed by a minimum duration.

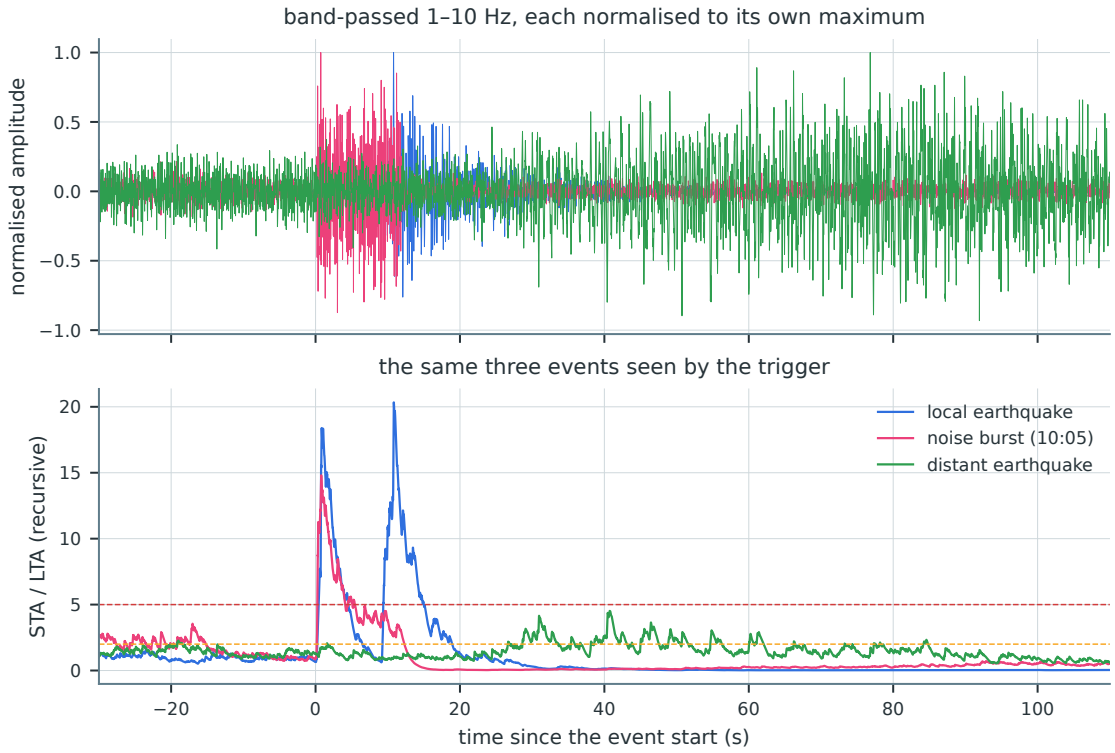


Figure 10.6: Three events seen by the trigger, each aligned at its start. The earthquake (blue) keeps a decaying coda; the burst (pink) stops after 12 s; the distant earthquake (green) never reaches the threshold because its energy lies below the 1–10 Hz pass band.

LTA contamination
: a long event raises the LTA and lowers the ratio for a while after it, so an aftershock may be missed. Some detectors *freeze* the LTA while a trigger is active.

Slow rise of noise
: the LTA lags a steadily rising noise level, which keeps $R > 1$; a rising wind can cause a false alarm.

Weak distant events
: emergent, long-period events lie outside a 1–10 Hz band; a detector for them needs a different band and longer windows (the sweep shows the distant earthquake lost already at $r = 5$).

10.8 Latency

How soon after an event begins does the trigger fire? Assume that the CF power jumps at $t = 0$ from the background P_0 to P_1 and that the LTA hardly changes in the first seconds. With the recursive STA of (10.2), whose time constant is τ_s,

$$\mathrm{STA}(t) = P_1 + (P_0 - P_1)\, e^{-t/\tau_s}. \tag{10.3}$$

The ratio reaches r_{on} when $\mathrm{STA} = r_{\mathrm{on}} P_0$, at the delay

$$t_d = -\tau_s \ln\!\left(\frac{P_1 - r_{\mathrm{on}} P_0}{P_1 - P_0}\right), \qquad P_1 > r_{\mathrm{on}} P_0. \tag{10.4}$$

Equation (10.4) has two consequences. A strong event ($P_1 \gg r_{\mathrm{on}} P_0$) triggers after a small fraction of τ_s: for a power increase of 100× and $r_{\mathrm{on}} = 5$, $t_d = 0.04\,\tau_s$. A weak event ($P_1 \to r_{\mathrm{on}} P_0$) takes arbitrarily long, and one with $P_1 \le r_{\mathrm{on}} P_0$ is never detected at all. (Appendix A.5 derives the formula.) The trigger time is therefore a *detection* time, not an onset time; for the onset the pass band must be causal and a dedicated picker must refine it.

10.9 Design of the RS3D alert pipeline

Design proposal

> The following is the design that is planned for the first implementation. It will be revised when the feature is built.

Goals. Detect events on the live stream without a browser open; tell the user through the channels they enabled; keep a permanent record; and never fail silently.

Architecture. The detector runs as a background service inside the application process, fed from the same ring buffer as the Live page (Chapter 11), so it keeps running when no browser tab is open. Figure 10.7 shows the stages and figure 10.8 the logic of one detector.

Event record. Each event stores the onset time, the duration, the peak STA/LTA ratio, the peak amplitude (in counts, and in m/s if the response is cached), the dominant frequency, the channel(s), and a snapshot of the waveform with its helicorder position. This is what the user sees in the notification and what the saved plot and log contain.

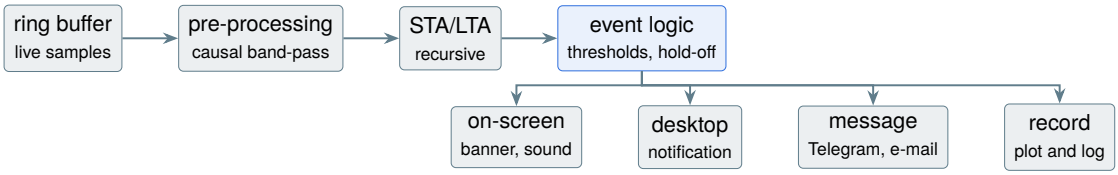


Figure 10.7: Proposed alert pipeline. Each output on the bottom row is an independent switch in Settings, and the record is always written.

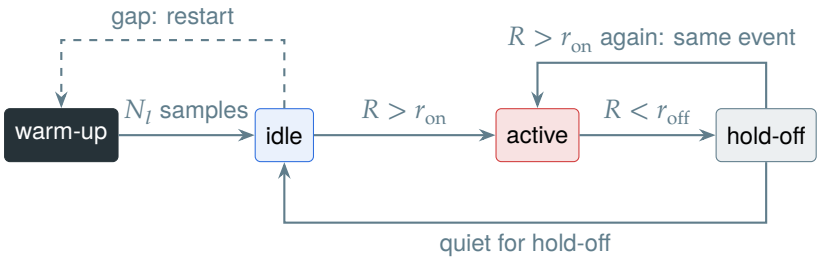


Figure 10.8: State of one detector. A new crossing during the hold-off period extends the same event, which is how the two trigger intervals of figure 10.1 become one event.

Proposed initial parameters. They come from the demo-day analysis above and are meant as a starting point to be tuned on real data from the user's site.

Known limitation. An energy trigger cannot tell an earthquake from a burst of equal power. The first release will report both, label an event "short, no coda" when it ends within a few seconds, and leave the decision to the user. The criteria of Section 10.7 are candidates for later versions.

Comparison with rsudp. The widely used *rsudp* software (Raspberry Shake S.A. 2026) also raises alerts from an STA/LTA ratio on a Raspberry Shake stream. Community examples use a 6 s STA and a 30 s LTA and thresholds between about 1 and 4, with

Table 10.3: Proposed initial alert parameters (design proposal).

Parameter	Initial value	Reason
Channel	EHZ	the vertical channel carries the P wave cleanly
Band-pass	1–10 Hz, causal 4th order	local events; matches the Live default
STA / LTA	1 s / 30 s	figures 10.1 and 10.2
r_{on} / r_{off}	5 / 2	zero false alarms on the demo day (figure 10.5)
Hold-off	30 s	merges the P and S trigger intervals
Minimum duration	2 s	rejects single-sample spikes
Warm-up	N_l/f_s and after every gap	the LTA is not valid before

the observation that suitable thresholds depend strongly on the filter band and on site noise; no authoritative list of its built-in defaults could be located while this manual was written, so none is quoted here. RS3D's alerts are intended to complement it, with the added record-keeping and the helicorder and detail views of the application.

11 Real-time data handling

The Live page and the future alert service both consume a continuous stream. This chapter describes how that stream is received, buffered and kept alive, and how the other path, the download of day files, works.

11.1 SeedLink

SeedLink is a protocol for distributing seismic data over TCP. A client connects to the server (on a Shake, port 18000), asks which stations and streams it offers (`INFO STREAMS`), selects the channels it wants, and then simply receives miniSEED records as they are completed by the digitiser. The application uses ObsPy's `EasySeedLinkClient` for the protocol.

In RS3D

`core/seedlink.py` first *probes* the Shake (connect, parse the XML answer of `INFO STREAMS`, disconnect) to learn the network, station, location and channel names, which is what the Status page and **Find Shake** use. The streaming class `SeedLinkSource` then selects `EHZ`, `EHN` and `EHE`. ObsPy reconnects by itself inside its collect loop, but its defaults (120 s idle timeout, 30 s delay) are far too slow for a live display, so the application lowers them to a 10 s network timeout and a 2 s delay. One workaround is documented in the source: the client is created with `autoconnect=False` and connected manually, because ObsPy 1.5.1 crashes when a timeout is set after an automatic connection.

11.2 The ring buffer

Incoming records are appended to a *ring buffer* that holds the most recent 10 minutes per channel. Records arrive in short pieces (typically a few seconds each); the buffer stores each channel as a list of *contiguous chunks*: a new piece is appended to the last chunk when it continues it exactly (same sampling rate, start within half a sample interval of the previous end) and otherwise starts a new chunk, so every chunk is gap-free by construction. Chunks that end more than 600 s before the newest data are discarded. All access is guarded by a lock, because the receiving thread writes while the display thread reads.

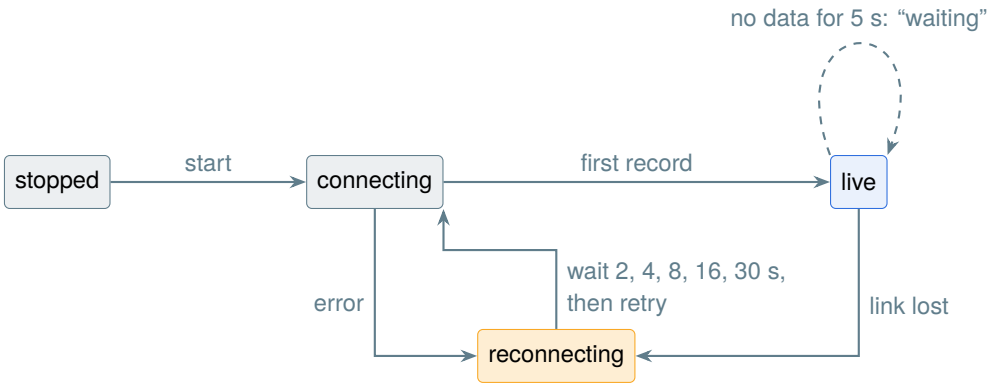


Figure 11.1: States of the live connection. The back-off sequence resets when a connection has delivered data.

To draw, the display asks for a *snapshot*: the samples of one channel in the window $[t_e - W, t_e]$, where t_e is the newest time and W the chosen window (30 s to 10 min). The snapshot is assembled from the chunks, with a `NaN` separator between chunks so that gaps stay visible. If a band-pass is requested it is applied to each chunk on its own (Chapter 4); then, if more than 1500 points remain, min/max decimation (Chapter 9) reduces them. The page redraws every 0.25 s, and **Pause** freezes the picture while the buffer continues to fill.

11.3 Keeping the link alive

Networks fail: a cable is unplugged, a Shake restarts. The streaming thread therefore runs a small state machine (figure 11.1). It starts in *connecting*; when the first record arrives it is *live*; when the connection ends or fails it waits and tries again in *reconnecting*. The wait grows with each consecutive failure along the sequence

$$\text{back-off} = 2,\ 4,\ 8,\ 16,\ 30 \text{ seconds} \quad (\text{then } 30 \text{ s}), \tag{11.1}$$

so a Shake that is off for an hour is not polled every two seconds, and the sequence restarts at 2 s as soon as a connection has delivered data. A second condition covers the case where the TCP connection survives but data stop: if no record has arrived for 5 seconds while the state is *live*, the page shows *waiting for data*. When a connection error is the reason, the reason text (for example *connection refused*) is shown next to *reconnecting*.

In RS3D

`RingBuffer`, `LiveStream` and the constants `BACKOFF`$= (2, 4, 8, 16, 30)$ are in `core/live.py`; the display loop is in `app/pages/live.py`. Data are not persisted by the live path: the Live page shows what the buffer holds; permanent data come

from the Shake's own archive through the Data page.

11.4 Downloading the archive

The Shake writes its own SDS archive. The application copies day files with SFTP over the same SSH login that the Status page uses. For each selected channel it lists the remote days, compares them with the days already on the laptop, and fetches the missing ones *plus the newest remote day*, which is still growing. A file is first written as `<name>.part` and renamed when complete, so a transfer cut by a cable fault leaves no corrupt day file. Reading is read-only on the Shake; the application never modifies it.

11.5 Latency

The delay between ground motion and the picture has three parts: the digitiser must complete a record before sending it (a few seconds), the network adds milliseconds on a direct cable, and the page refreshes every 0.25 s. A practical latency of a few seconds is therefore expected, and is set by the record length, not by the application's processing.

12 Software architecture

This chapter describes how the code is organised, so that the algorithms of the preceding chapters can be found, tested and changed.

12.1 Two layers

The package `rs3d` has two layers with a strict dependency direction.

`core/` every algorithm in this manual. These modules know nothing of the browser; they take and return ObsPy streams, NumPy arrays and plain data classes, which makes them testable in milliseconds with synthetic signals.

`app/` presentation. The chart builders in `app/charts.py` are *pure functions* that return the option dictionaries of the charting library, so even the shape of a plot is unit-tested without a browser.

12.2 Where each algorithm lives

Table 12.1 maps every topic of this manual to the module and the main functions that implement it.

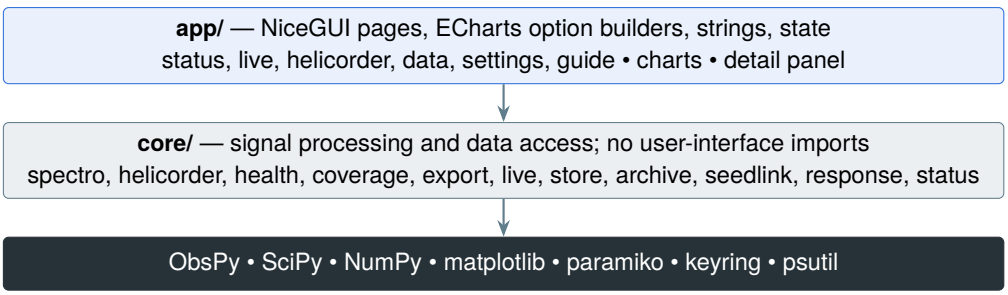


Figure 12.1: Layers of the application. Arrows point from a layer to the layer it uses.

Table 12.1: Algorithm → module.

Topic	Module	Main functions
Day-file layout, import	`core/store.py`	`day_path`, `load`, `import_file`
Response, units, filtering	`core/response.py`, `export.py`	`get_inventory`, `process`
Spectra, spectrogram, noise	`core/spectro.py`	`amplitude_spectrum`, `spectrogram`, `window_for`, `psd_summary`
Completeness, coverage	`core/health.py`, `coverage.py`	`health`, `coverage`
Helicorder rows and windows	`core/helicorder.py`	`build_rows`, `scale_for`, `click_to_time`, `zoom_window`, `box_window`
Decimation, live buffer	`core/live.py`	`minmax_decimate`, `RingBuffer`, `LiveStream`
Shake access	`core/seedlink.py`, `shell.py`, `archive.py`	`probe`, `SeedLinkSource`, `sync`
Plot shapes	`app/charts.py`	`helicorder_options`, `spectrogram_options`, `psd_options`
Detail view, clipboard	`app/detail.py`, `core/export.py`	`DetailPanel`, `write_clipboard_png`

12.3 Reproducibility by construction

The Data page can show, and save, a short Python program that reproduces exactly the processing behind a preview: it reads the same day files, applies the same response removal and the same band-pass, and prints the stream. The program is generated by `core/codegen.py` from the same `Selection` object that the application itself uses, so the code and the screen cannot drift apart. A researcher can therefore take any number off the screen and recompute it outside the application.

12.4 Testing

Three levels of test protect the algorithms.

1. **Unit tests** of `core/` with synthetic data: a sine must peak in its own bin and read its own amplitude, a spike must survive decimation, a gap must stay a gap, a click must map to its row and time.

2. **Page tests** with NiceGUI's user simulation: each page renders with a fake device; controls change state as expected.

3. **Real-browser tests** (Playwright with Google Chrome, marker `e2e`): import a file, open the helicorder, click a row, zoom with a dragged rectangle, open the clip-

board and download the picture. Each such test has been shown to fail when the behaviour it protects is deliberately broken.

A fourth marker, `device`, selects tests that need a real Shake and is skipped by default.

12.5 Configuration and storage

All files live in one folder (on Linux `~/.local/share/rs3d`, overridable with the `RS3D_DATA_DIR` environment variable): `config.toml` (devices), `settings.toml` (time zone, last station, units and band-pass), `known_hosts`, `sds/` (data), `response/` (StationXML cache), `logs/`. Passwords are kept in the operating system's key store through `keyring`, never in these files.

A Derivations

A.1 Scaling of the amplitude spectrum

Let $x[n] = A\cos(\theta n)$ with $\theta = 2\pi f_0/f_s$, and let the window be $w[n]$, $n = 0, \dots, N-1$. Write $x[n] = \frac{A}{2}(e^{\mathrm{j}\theta n} + e^{-\mathrm{j}\theta n})$. The windowed DFT at a bin with $2\pi k/N = \theta$ is

$$X_w[k] = \sum_n w[n]\,x[n]\,e^{-\mathrm{j}\theta n} = \frac{A}{2}\sum_n w[n] \;+\; \frac{A}{2}\sum_n w[n]\,e^{-\mathrm{j}2\theta n}. \tag{A.1}$$

The second sum is the window's transform at twice the tone frequency, far outside the main lobe, and is negligible for a tapered window. Hence $|X_w[k]| \approx \frac{A}{2}\sum_n w[n]$, which is Eq. (5.4). If only a subset $\mathcal{U}$ of the samples is valid and the others are set to zero, the first sum runs over $\mathcal{U}$ and the scale becomes $\sum_{n\in\mathcal{U}} w[n]$, the rule used by the application.

A.2 Hann window constants

For $w[n] = \frac{1}{2}(1 - \cos\frac{2\pi n}{N-1})$ and large N, the mean over a full period of the cosine is zero, so $\sum w \approx N/2$, giving a coherent gain of $\frac{1}{2}$. For the energy, with $c = \cos\frac{2\pi n}{N-1}$ averaging to 0 and c^2 to $\frac{1}{2}$,

$$\sum w^2 \approx N\,\overline{\tfrac{1}{4}(1 - 2c + c^2)} = \tfrac{N}{4}(1 + \tfrac{1}{2}) = \tfrac{3N}{8}.$$

The equivalent noise bandwidth is therefore

$$\mathrm{ENBW} = \frac{N\sum w^2}{(\sum w)^2} = \frac{N \cdot 3N/8}{N^2/4} = 1.5 \text{ bins},$$

matching Table 5.1.

A.3 Zero-phase filtering

Let a filter have impulse response h and frequency response $H(\omega)$. Filtering forward gives $Y_1 = HX$. Time reversal of a real sequence turns its transform into the complex conjugate (up to a linear phase that is undone by the second reversal), so the backward pass multiplies by H^* and the final reversal restores the time direction: $Y = H^*H\,X = |H|^2\,X$. The magnitude response in decibels is $20\log_{10}|H|^2 = 2 \cdot 20\log_{10}|H|$, twice that of one pass, so the -3 dB corner of one pass is a -6 dB corner of the zero-phase filter.

A.4 The recursive averages

Rearranging Eq. (10.2): $\mathrm{STA}_i = (1 - \frac{1}{N})\mathrm{STA}_{i-1} + \frac{1}{N}c[i]$. Unrolling,

$$\mathrm{STA}_i = \frac{1}{N}\sum_{k=0}^{i}\left(1 - \frac{1}{N}\right)^{k} c[i-k] + \left(1 - \frac{1}{N}\right)^{i+1}\mathrm{STA}_{-1}. \tag{A.2}$$

The weights sum to $1 - (1 - 1/N)^{i+1} \to 1$ and, for large N, $(1 - 1/N)^k \approx e^{-k/N}$: an exponential window of time constant N samples. The initial-condition term decays with the same time constant, which is why a warm-up of N_l samples is needed.

A.5 Detection delay

For a step of the CF power from P_0 to P_1 at $t = 0$ the continuous-time analogue of (10.2) is $\dot{S} = (P_1 - S)/\tau_s$ with $S(0) = P_0$, whose solution is $S(t) = P_1 + (P_0 - P_1)e^{-t/\tau_s}$. Setting $S(t_d) = r_{\mathrm{on}}P_0$ and solving for t_d,

$$e^{-t_d/\tau_s} = \frac{P_1 - r_{\mathrm{on}}P_0}{P_1 - P_0} \;\Rightarrow\; t_d = -\tau_s \ln\frac{P_1 - r_{\mathrm{on}}P_0}{P_1 - P_0},$$

which is Eq. (10.4). The assumption that the LTA is constant during t_d holds when $t_d \ll \tau_l$; for $\tau_l = 30$ s and $\tau_s = 1$ s it is good for all events strong enough to trigger within a few seconds.

A.6 The F distribution of the ratio

If the CF is the square of zero-mean Gaussian noise of variance σ^2 filtered to bandwidth B, the average of c over T seconds is $\sigma^2\chi^2_\nu/\nu$ with $\nu \approx 2BT$ degrees of freedom (the number of independent samples is about $2BT$, by the sampling theorem). The STA and LTA are independent only if their windows do not overlap, which the classic form approximately satisfies, and then $R = \dfrac{\chi^2_{\nu_s}/\nu_s}{\chi^2_{\nu_l}/\nu_l} \sim F_{\nu_s,\nu_l}$. The tail probabilities of Table 10.1 are $1 - F_{\nu_s,\nu_l}(r)$ with $\nu_s = 18$, $\nu_l = 540$. They apply to *one* independent evaluation; the number of false alarms per day also depends on how many independent evaluations occur and on how exceedances cluster, which is why the table reports simulated counts alongside the tail.

A.7 Error of min/max decimation

Let a bin contain samples $x[n]$, $n \in b$, and let $\hat{x}$ denote the drawn polyline through the kept extremes. Every sample of the bin lies between its minimum and maximum, so the vertical distance between any sample and the plotted bin is at most $x_b^{\max} - x_b^{\min}$, and the horizontal position of the extremes is exact. Therefore the plotted envelope

$$[\, x_b^{\min},\ x_b^{\max} \,]$$

equals the true envelope of the bin, and the only information lost is the order of the samples between the extremes.

B Parameter reference

Every numerical constant that shapes a result, with the file in which it is set (relative to `rs3d-app/src/rs3d/`). Table B.1 covers acquisition and conditioning, table B.2 analysis and display, and table B.3 the health checks.

Table B.1: Acquisition, buffering and conditioning.

Quantity	Value	File
Sampling rate (RS3D)	100 Hz	device
SeedLink port	18000	`core/models.py`
SeedLink network timeout / delay	10 s / 2 s	`core/seedlink.py`
Ring buffer length	600 s	`core/live.py`
Reconnect back-off	2, 4, 8, 16, 30 s	`core/live.py`
Stale-data warning	5 s	`core/live.py`
Live refresh interval	0.25 s	`app/pages/live.py`
Live windows	30, 60, 300, 600 s	`app/pages/live.py`
Live decimation limit	1500 points	`app/pages/live.py`
Live band-pass default / order	1–10 Hz / 2 (padded forward–backward)	`app/pages/live.py`, `core/live.py`
Live upper-corner limit	$0.45 f_s$	`core/live.py`
Export band-pass	Butterworth order 4, two passes, demeaned	`core/export.py`
Response removal	ObsPy defaults: water level 60 dB, 5 % taper	`core/export.py`

Table B.2: Spectral analysis, helicorder and detail view.

Quantity	Value	File
Amplitude-spectrum window	Hann, scale $2/\sum w$	`core/spectro.py`
Minimum samples for a spectrum	8	`core/spectro.py`
Spectrogram window	SciPy default Tukey ($\alpha = 0.25$)	`core/spectro.py`
Spectrogram overlap	50 %	`core/spectro.py`
Window-length rule	$T/360$, limited to 1–600 s	`core/spectro.py`
Noise-spectrum percentiles	10, 50, 90	`core/spectro.py`
Spectrogram heat-map size	at most 400 × 200 cells	`app/charts.py`
Spectrogram colour range	2nd–98th percentile	`app/charts.py`
Data-page waveform thinning	stride to about 4000 points	`app/charts.py`
Helicorder row length / rows	30 min / 48	`core/helicorder.py`
Points per helicorder row	3000	`core/helicorder.py`
Helicorder amplitude scale	$2P_{99}/F$, $F \in \{0.5, 1, 2, 4, 8\}$	`core/helicorder.py`
Helicorder row height / click grid	14 px / every 10 s	`app/charts.py`
Detail window, default	120 s	`app/detail.py`
Detail window limits	5 s – 3600 s	`core/helicorder.py`
Zoom factors / shift	×2, $\times\frac{1}{2}$ / $\pm\frac{1}{2}$ span	`core/helicorder.py`
Box-zoom amplitude rounding	4 significant figures	`app/detail.py`

Table B.3: Data health and Shake checks.

Quantity	Value	File
Gap merge tolerance (coverage, health)	$0.5/f_s$	`core/coverage.py`, `core/health.py`
Helicorder gap break	$1.5/f_s$	`core/helicorder.py`
Gaps listed on the Data page	at most 50	`app/pages/data.py`
Clock offset: warning / error	$>$ 100 ms / $>$ 1000 ms	`core/status.py`
Free storage warning	$<$ 10 %	`core/status.py`
Link-local address range	169.254.0.0/16	`core/netcheck.py`
Archive: day at risk	not downloaded and within one day of the retention limit	`core/status.py`

References

Allen, R. V. (1978). "Automatic earthquake recognition and timing from single traces". In: *Bulletin of the Seismological Society of America* 68.5, pp. 1521–1532.

Beyreuther, M., R. Barsch, L. Krischer, T. Megies, Y. Behr, and J. Wassermann (2010). "ObsPy: A Python toolbox for seismology". In: *Seismological Research Letters* 81.3, pp. 530–533. DOI: 10.1785/gssrl.81.3.530.

Butterworth, S. (1930). "On the theory of filter amplifiers". In: *Experimental Wireless and the Wireless Engineer* 7, pp. 536–541.

Gabor, D. (1946). "Theory of communication". In: *Journal of the Institution of Electrical Engineers* 93.26, pp. 429–457.

Harris, F. J. (1978). "On the use of windows for harmonic analysis with the discrete Fourier transform". In: *Proceedings of the IEEE* 66.1, pp. 51–83.

Havskov, J. and G. Alguacil (2016). *Instrumentation in Earthquake Seismology*. 2nd ed. Springer.

International Federation of Digital Seismograph Networks (2012). *SEED Reference Manual, Version 2.4*.

Krischer, L. et al. (2015). "ObsPy: a bridge for seismology into the scientific Python ecosystem". In: *Computational Science & Discovery* 8.1, p. 014003. DOI: 10.1088/1749-4699/8/1/014003.

McNamara, D. E. and R. P. Buland (2004). "Ambient noise levels in the continental United States". In: *Bulletin of the Seismological Society of America* 94.4, pp. 1517–1527. DOI: 10.1785/012003001.

Peterson, J. (1993). *Observations and modeling of seismic background noise*. Open-File Report 93-322. U.S. Geological Survey.

Raspberry Shake S.A. (2026). *rsudp: continuous Raspberry Shake UDP data consumer*. URL: https://github.com/raspishake/rsudp (visited on 10/15/2026).

Steinarsson, S. (2013). "Downsampling Time Series for Visual Representation". MSc thesis. University of Iceland.

Trnkoczy, A. (2012). "Understanding and parameter setting of STA/LTA trigger algorithm". In: *New Manual of Seismological Observatory Practice 2 (NMSOP-2)*. Ed. by P. Bormann. Information Sheet IS 8.1. Potsdam: Deutsches GeoForschungsZentrum GFZ. DOI: 10.2312/GFZ.NMSOP-2_IS_8.1.

Welch, P. D. (1967). "The use of fast Fourier transform for the estimation of power spectra: a method based on time averaging over short, modified periodograms". In: *IEEE Transactions on Audio and Electroacoustics* 15.2, pp. 70–73.

Withers, M. et al. (1998). "A comparison of select trigger algorithms for automated global seismic phase and event detection". In: *Bulletin of the Seismological Society of America* 88.1, pp. 95–106.